Universidad Católica San Pablo

FACULTAD DE INGENIERÍA Y COMPUTACIÓN

DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

Escuela Profesional de Ciencia de la Computación

Arquitectura basada en representación intermedia para la generación confiable de consultas SQL/PGQ desde lenguaje natural

Tesis

Presentada por el bachiller

Guido Luis Tapia Oré

Para Optar por el Título profesional de:

Licenciado en Ciencia de la Computación

Asesor: Dra. Regina Paola Ticona Herrera

** Arequipa, Julio 2026 **

A mi familia por su incondicional apoyo en este proceso, a los profesores que fueron una guía en este proceso de investigación, a mis compañeros que ayudaron a no rendirme en este proceso.

Abreviaturas

LLM Large Language Model

PGQ Property Graph Queries

RBAC Role-Based Access Control

RDBMS Relational Database Management System

RI Representación Intermedia

SQL Structured Query Language

Agradecimientos

Aquí deberás colocar a quien y porque agradeces. Ejemplo:

En primer lugar deseo agradecer a Dios por haberme guiado a lo largo de estos cinco años de estudio.

Agradezco a mis padres por el apoyo brindado para forjarme como un profesional.

Agradezco a la universidad, mi *alma matter*, por haberme cobijado y brindado la formación que ahora me permitirá ayudar a construir una mejor sociedad.

Agradezco de forma muy especial a mi asesor Prof. Dr./Mag. nombre 1 por haberme guiado en esta tesis. ...

Deseo agradecer de forma especial a mis profesores: nombre 1, nombre 2, nombre 3 porque fueron ejemplos que deseo seguir en mi vida profesional.

Deseo agradecer al personal administrativo de la universidad: nombre 1, nombre 2, nombre 3. Muchas gracias por la atención brindada y porque siempre estuvieron dispuestas a ayudarnos.

Resumen

Aquí deberás colocar hasta 300 palabras como máximo, describiendo el problema que intentas resolver, la justificación, aportes o soluciones que planteas, qué tanto los haz alcanzado en calidad de resultados obtenidos.

Abstract

Here you should enter up to 300 words maximum, describing the problem you are trying to solve, the justification, contributions or solutions you are proposing, how much you have achieved them in terms of obtained outcomes.

Índice general

1. Introducción **2**

1.1. Motivación y Contexto 2

1.2. Planteamiento del Problema 4

1.3. Objetivos 5

1.3.1. Objetivos Específicos 5

1.4. Organización de la tesis 5

1.5. Cronograma 6

2. Fundamentos de Generación Text-to-Query y Consultas Híbridas **8**

2.1. Modelos de lenguaje y generación estructurada 8

2.1.1. Arquitectura transformer y decodificación autorregresiva 9

2.1.2. Prompting y aprendizaje en contexto 10

2.2. Modelos de datos y lenguajes de consulta 10

2.2.1. Modelo relacional y álgebra relacional 10

2.2.2. SQL como lenguaje declarativo 11

2.2.3. Grafos de propiedad y pattern matching 11

2.2.4. El estándar SQL/PGQ como integración 12

2.3. Semantic parsing y generación *text-to-query* 12

2.3.1. *Semantic parsing*: definición y marco general 12

2.3.2. *Text-to-SQL* como caso particular y extensión a *text-to-query* 12

2.3.3. *Schema linking* y *grounding* al esquema 13

2.4. Representaciones intermedias 14
2.4.1. Noción formal de representación intermedia 14
2.4.2. Propiedades deseables 14
2.4.3. Separación entre generación estadística y compilación determinista . 14
2.5. Gramáticas formales y decodificación restringida 15
2.5.1. Gramáticas formales y jerarquía de Chomsky 15
2.5.2. Decodificación restringida como acoplamiento generación–parser . . 15
2.5.3. Sistemas de tipos y reglas locales de validación 15
2.5.4. Alcance y límites formales 16
2.6. Verificación de consultas generadas 16
2.6.1. Verificación estática 16
2.6.2. Verificación dinámica y oráculos parciales 16
2.6.3. Testing metamórfico 17
2.6.4. Bucles de refinamiento 17

3. Arquitecturas y Enfoques Actuales en Text-to-Query 18
3.1. Marco taxonómico de análisis 18
3.2. Pipelines por etapas y representaciones intermedias 19
3.3. Sistemas multi-agente y arquitecturas agénticas con búsqueda 21
3.4. Decodificación restringida y verificación post-generación 23
3.5. Robustez del esquema y entornos empresariales 24
3.6. Generación de consultas híbridas sobre el estándar SQL/PGQ 25
3.7. Criterios de evaluación y señales empíricas 25
3.8. Síntesis, tensiones y vacíos de investigación 26

4. Propuesta: Arquitectura basada en Representación Intermedia para Generación Confiable de Consultas SQL/PGQ 28
4.1. Requisitos y principios de diseño 29
4.2. Flujo metodológico general 30

4.3. Arquitectura y componentes 32

4.4. Implementación sobre la base experimental 32

4.5. Justificación de decisiones de diseño 32

4.6. Síntesis y transición 32

5. Experimentos **33**

6. Resultados **34**

7. Conclusiones y Trabajos Futuros **35**

7.1. Problemas encontrados 35

7.2. Recomendaciones 35

7.3. Trabajos futuros 35

Bibliografía **40**

Índice de tablas

1.1. Cronograma de actividades de la tesis. 6

3.1. Vector taxonómico de los trabajos centrales revisados, proyectado sobre los seis ejes definidos en la Sección 3.1. La última fila ubica la propuesta de la presente tesis. Las celdas con guion ("—") indican que la dimensión no es objetivo declarado del trabajo o no es aplicable. 20

Índice de figuras

4.1. Flujo metodológico general del *pipeline* propuesto. Las cinco etapas secuenciales procesan la pregunta en lenguaje natural y el esquema dual (relacional y grafo de propiedad) hasta producir una consulta Structured Query Language (SQL)/Property Graph Queries (PGQ) verificada. 31

Capítulo 1

Introducción

Este capítulo establece el contexto, la motivación y el problema de investigación que aborda la presente tesis, cuyo enfoque central radica en el diseño de una arquitectura de razonamiento estructural, basada en Representación Intermedia (RI), para mitigar las alucinaciones en la generación *text-to-query* de consultas empresariales complejas, específicamente orientada al estándar híbrido SQL/PGQ.

Esta investigación se sitúa en la intersección entre los Large Language Models (LLMs), los sistemas de gestión de bases de datos relacionales (Relational Database Management Systems (RDBMSs)) y los lenguajes de consulta híbridos, pues estos lenguajes combinan el paradigma relacional tradicional con las consultas sobre grafos de propiedades (*property graphs*), un modelo recientemente formalizado en estándares internacionales como ISO/IEC [ISO/IEC JTC 1/SC 32, 2023].

Como se evidenciará en el posterior desarrollo, el diseño propuesto representa una respuesta estructurada al creciente interés de la industria por desplegar interfaces en lenguaje natural altamente fiables en entornos empresariales a gran escala. Reducir las alucinaciones y mantener la consistencia lógica constituyen retos críticos actuales [Chi, 2025, Qu et al., 2024], los cuales demandan métodos avanzados que integren el razonamiento tabular y de grafos [Stoisser et al., 2025].

1.1. Motivación y Contexto

En el contexto actual de la adopción de LLMs en procesos empresariales y el creciente interés por integrarlos en la optimización de flujos de trabajo, se evidencia una clara tendencia hacia interfaces basadas en lenguaje natural [Adedeji and Mabere, 2023], en detrimento de las interfaces gráficas tradicionales. Este cambio es particularmente notable en organizaciones *data-driven*, cuya toma de decisiones depende de la capacidad de responder preguntas de negocio en ciclos cortos y con altas garantías de calidad [Pérez et al., 2025]. En este marco, las interfaces en lenguaje natural para la consulta de datos han resurgido con fuerza, habilitando experiencias de *self-service analytics*. Estas permiten que perfiles no técnicos formulen preguntas en texto libre y lenguaje cotidiano,

obteniendo resultados estructurados sin la necesidad de escribir manualmente consultas complejas [Quamar et al., 2022].

Sin embargo, el despliegue de estos modelos en entornos de producción demanda el cumplimiento de tres pilares críticos: (i) *integridad técnica*, que garantiza tanto la ejecutabilidad en el RDBMS como la fidelidad semántica de la consulta [Lei et al., 2025]; (ii) *seguridad y gobernanza*, mediante mecanismos de control de acceso basado en roles (Role-Based Access Control (RBAC)) y auditoría para prevenir la exfiltración de datos sensibles [Liu et al., 2025]; y (iii) *robustez y escalabilidad del esquema*, asegurando que el modelo mantenga su precisión frente a catálogos de datos masivos, dinámicos y con nomenclaturas complejas [Tang et al., 2026].

Asimismo, a pesar de los avances acelerados, los LLMs aún presentan fallas sistemáticas de *fidelidad y grounding* al generar artefactos formales. Específicamente en la tarea de *text-to-query*, estas fallas se manifiestan como referencias a tablas o columnas inexistentes, rutas de *join* inventadas, condiciones lógicas mal interpretadas o agregaciones incongruentes con el esquema. Si bien esta es una limitación general de los modelos de lenguaje, el riesgo se agrava severamente cuando el código se genera para un RDBMS: un error no solo afecta la exactitud analítica, sino que puede inducir sobrecostos computacionales, vulnerabilidades de seguridad e incluso modificar datos existentes. Investigaciones recientes demuestran que el alto desempeño en *benchmarks* clásicos decae drásticamente al enfrentarse a bases de datos corporativas reales, ruidosas y con flujos de trabajo de múltiples pasos [Li et al., 2023, Lei et al., 2025], lo cual refuerza la necesidad de transitar desde enfoques de *prompting* monolíticos y arquitecturas puramente agénticas hacia sistemas más robustos y orquestados [Fürst et al., 2024].

A su vez en paralelo, el ecosistema de bases de datos está evolucionando hacia modelos híbridos. En dominios como prevención de fraude, ciberseguridad, logística y *compliance*, el modelado de relaciones mediante grafos resulta más natural para expresar caminos y vecindarios complejos. No obstante, dado que las bases de datos relacionales siguen dominando el entorno corporativo por su estabilidad, la integración de *property graphs* se presenta como una evolución natural de la industria. Esto se valida con la reciente estandarización ISO/IEC 9075-16:2023 (SQL/PGQ), la cual formaliza la consulta de grafos de propiedades directamente desde SQL [ISO/IEC JTC 1/SC 32, 2023]. Si bien esto unifica ambos paradigmas, también incrementa significativamente la dificultad de la tarea *text-to-query*: el espacio de búsqueda se expande y la validación lógica se vuelve más rigurosa, multiplicando el riesgo de alucinaciones.

En la industria actual, las soluciones comerciales suelen apoyarse en estrategias heurísticas como inyectar el esquema en el *prompt* o aplicar ciclos de autocorrección mediante retroalimentación en texto libre. Estos métodos han demostrado ser frágiles ante la ambigüedad semántica del usuario y el *schema drift* [Ding et al., 2026]. Por su parte, la literatura reciente aborda este reto desde diversos frentes: decodificación restringida por gramática para garantizar validez sintáctica [Scholak and et al., 2022], arquitecturas modulares que dividen la carga de razonamiento [Pourreza and Rafiei, 2023], técnicas de alineación pre-generación para mitigar alucinaciones [Qu et al., 2024], métodos de evaluación sin *ground truth* como el *metamorphic testing* [Chi, 2025], y la unificación del razonamiento tabular y de grafos [Stoisser et al., 2025], entre otros.

Estos antecedentes convergen en una observación central: si bien existen técnicas parciales para mitigar alucinaciones en *text-to-SQL* y multiples propuestas dentro de esta área, aún existe un gran campo para investigar su implementación de forma sistémica y mejora, especialemente orientdo a la integración con el estándar híbrido SQL/PGQ. La ausencia de una arquitectura que coordine estas técnicas sobre un esquema dual (relacional y de grafo) deja sin resolver el problema de fidelidad en entornos empresariales donde coexisten ambos paradigmas. Es esta brecha la que constituye la principal motivación de la presente investigación.

1.2. Planteamiento del Problema

La generación de consultas a bases de datos a partir de lenguaje natural exige resolver un problema de traducción entre dos espacios formalmente distintos: el espacio continuo y ambiguo del texto libre y el espacio discreto de un lenguaje de consulta con semántica determinista, restringido por un esquema de base de datos específico. En entornos empresariales, donde las decisiones operativas dependen de la corrección de estas consultas, el problema adquiere relevancia práctica directa, dado que una consulta incorrecta no solo produce resultados erróneos, sino que puede traducirse en sobrecostos computacionales y comprometer la integridad de los datos.

En el contexto de la creciente integración de soluciones basadas en LLMs, estos modelos, al operar mediante predicción autorregresiva de tokens, carecen de mecanismos intrínsecos que vinculen la secuencia generada con las restricciones formales del esquema subyacente ni con la semántica específica del lenguaje de consulta objetivo. Esta desalineación entre la generación estadística y la corrección formal constituye una de las causas centrales de las denominadas *alucinaciones* en *text-to-query*: los modelos generan secuencias plausibles pero semánticamente inválidas respecto al esquema. En la práctica, esto se manifiesta como referencias a tablas o columnas inexistentes, uniones inválidas, condiciones lógicas inconsistentes o patrones de grafo mal formados.

El problema se agrava al considerar el estándar híbrido SQL/PGQ, donde la generación debe operar simultáneamente sobre dos estructuras formales superpuestas: el esquema relacional, el cual incluye tablas, columnas, restricciones, y el grafo de propiedad que representaría los nodos, aristas, propiedades clave-valor. En este contexto, los errores no solo afectan a cada estructura por separado, sino que surgen nuevas categorías de fallo derivadas de la incoherencia cruzada entre ambas. Esta dualidad contribuye a incrementar significativamente el espacio de decisiones del modelo y, con ello, la dificultad de garantizar la corrección de las consultas generadas.

Si bien la literatura reciente ha propuesto técnicas parciales para abordar distintas facetas de este problema, lo que se evidencia es un claro declive de rendimiento de las propuestas cuando son evaluadas con datasets de consultas híbridas SQL/PGQ. Esta brecha motiva la pregunta central de la presente investigación: *¿cómo mitigar de forma sistemática las alucinaciones estructurales y semánticas en la generación* text-to-query *sobre esquemas híbridos SQL/PGQ, de modo que las consultas resultantes sean correctas, verificables y operables en contextos empresariales?*

1.3. Objetivos

Objetivo general: Proponer una arquitectura de razonamiento estructural basada en RI para la generación *text-to-query* orientada al estándar SQL/PGQ, que mitigue las alucinaciones de esquema y los errores semánticos en escenarios de consulta empresarial.

1.3.1. Objetivos Específicos

- Caracterizar los tipos de alucinación en la generación *text-to-query* mediante una taxonomía de errores aplicable a consultas relacionales e híbridas, identificando sus causas computacionales.
- Diseñar una RI unificada para consultas híbridas SQL/PGQ que integre componentes relacionales y de patrón de grafo con reglas de tipado y compilación determinista.
- Implementar un *pipeline* de razonamiento estructural que incorpore anclaje al esquema, generación restringida por gramática y un bucle de verificación estática y dinámica.
- Evaluar experimentalmente la arquitectura propuesta frente a métodos del estado del arte mediante métricas de corrección, tasa de alucinación, robustez y eficiencia sobre *benchmarks* establecidos.

1.4. Organización de la tesis

El resto de este documento está organizado de la siguiente manera:

- El capítulo 2 presenta los fundamentos conceptuales y técnicos: LLMs y alucinaciones, *text-to-SQL*, el estándar SQL/PGQ, y las técnicas de RI, decodificación restringida y verificación.
- El capítulo 3 revisa las arquitecturas y enfoques actuales en *text-to-query* (pipelines por etapas, agentes, *constrained decoding*, detección y mitigación de alucinaciones) y posiciona el problema en el contexto empresarial y del estándar híbrido.
- El capítulo 4 describe la propuesta: la RI tipada para SQL/PGQ, el compilador de RI a SQL/PGQ, el pipeline de razonamiento estructural y el bucle de verificación (estático y por ejecución).
- El capítulo 5 detalla el diseño experimental: datasets/benchmarks, construcción de corpus híbrido para SQL/PGQ, métricas de alucinación y corrección, baselines y *ablations*.
- El capítulo 6 presenta y analiza los resultados obtenidos, incluyendo robustez a *schema drift*, seguridad/eficiencia, y evaluación de componentes, validando los resultados con datasets relevantes.

- El capítulo 7 concluye la tesis, resume aportes, limitaciones y líneas de trabajo futuro.

1.5. Cronograma

Tabla 1.1: Cronograma de actividades de la tesis.

		N.°	Actividad	Detalles	Semana	Fecha	%
Semestre 2026-1	Parcial	1	Alineamiento inicial del trabajo	Revisión del curso, entregables, estructura de la tesis y ajuste del tema con enfoque en SQL/PGQ + representación intermedia.	Sem. 1	11/03 – 14/03	2
		2	Definición final de tema y asesor	Alcance, contribución central y validación con asesora. Primera revisión de tabla survey.	Sem. 2	14/03 – 21/03	5
		3	Formalización y delimitación	Compromiso de asesoría, planteamiento del problema, objetivo general y específicos.	Sem. 3	21/03 – 28/03	10
		4	Consolidación del survey	Depuración del estado del arte, clasificación de trabajos, taxonomía y bibliografía base.	Sem. 4	28/03 – 01/04	15
		5	Redacción de Introducción v1	Motivación, contexto, problema, objetivos, organización y cronograma.	Sem. 5	01/04 – 08/04	18
		6	Cierre de Introducción	Ajustes con feedback y versión entregable del capítulo Introducción.	Sem. 5	08/04 – 11/04	20
		7	Redacción de Marco Teórico	Fundamentos teóricos, conceptos clave y modelos explicativos de la investigación.	Sem. 6	11/04 – 14/04	28
		8	Redacción de Estado del Arte	Literatura reciente, trabajos previos, vacíos de investigación y posicionamiento.	Sem. 6	14/04 – 18/04	35
		9	Capítulo Propuesta v1	Arquitectura, pipeline, representación intermedia, componentes y flujo de verificación.	Sem. 7	18/04 – 25/04	45
		10	Capítulo Propuesta v2	Protocolo experimental, datasets, métricas, baselines y criterios de evaluación.	Sem. 8	26/04 – 02/05	50
		11	Integración del Plan de Tesis	Consolidación de capítulos, coherencia interna, flujo narrativo y bibliografía final.	Sem. 9	03/05 – 09/05	55
		12	Plan de Tesis listo para envío	Cierre de documento, diapositivas y ensayo previo a la exposición.	Sem. 10	11/05 – 16/05	55
	Final	13	Implementación base experimental	Repositorio, estructura del prototipo, casos de prueba y baseline(s).	Sem. 11	17/05 – 23/05	62
		14	Implementación arquitectura v1	Flujo NL → RI → consulta estructurada/verificable.	Sem. 12	24/05 – 30/05	68
		15	Integración end-to-end y validación	Integración de componentes, validaciones estructurales y depuración.	Sem. 13	31/05 – 06/06	73
		16	Experimentación preliminar	Pruebas comparativas, recopilación de resultados y análisis preliminar.	Sem. 14	07/06 – 13/06	77
		17	Resultados al 80 y Avances	Prototipo funcional, resultados parciales sólidos y figuras iniciales.	Sem. 15	14/06 – 20/06	80

Continúa en la siguiente página...

		N.°	Actividad	Detalles	Semana	Fecha	%
		18	Cierre del documento de Avances	Documento completo, conclusiones preliminares y preparación de diapositivas.	Sem. 16	21/06 – 27/06	80
		19	Envío y exposición de Avances	Presentación final del semestre con resultados parciales consolidados.	Sem. 17	28/06 – 04/07	80
Int.		20	Revisión y fortalecimiento	Análisis de trabajos complementarios y refinamiento de la arquitectura propuesta.		10/07 – 10/08	82
Semestre 2026-2	Parcial	21	Proyecto de Investigación	Propuesta formal: impacto, viabilidad y posible aplicación institucional.	Sem. 1–2	12/08 – 22/08	84
		22	Ampliación experimental	Nuevos escenarios, ablations, análisis de errores y validación empresarial.	Sem. 3–5	23/08 – 15/09	87
		23	Resultados finales y discusión	Comparación con baselines, análisis crítico y discusión de aportes.	Sem. 6–8	16/09 – 06/10	90
		24	Preparación y Exposición Parcial	Presentación con metodología, resultados preliminares y validación ante jurado.	Sem. 9	07/10 – 13/10	91
	Final	25	Revisión y ajustes post-parcial	Correcciones metodológicas y refinamiento con feedback del jurado.	Sem. 10	14/10 – 20/10	92
		26	Síntesis de resultados y divulgación	Hallazgos en formato de reporte técnico o borrador de artículo.	Sem. 11	21/10 – 27/10	93
		27	Redacción final de tesis	Cierre de capítulos, conclusiones, trabajo futuro y referencias.	Sem. 12–14	28/10 – 15/11	96
		28	Correcciones con asesora	Ajustes de fondo y forma, normalización y aprobación final.	Sem. 15–16	16/11 – 30/11	98
		29	Preparación y Exposición Final	Diapositivas, ensayo de tiempos y sustentación final ante jurado.	Sem. 17–18	01/12 – 12/12	100

Capítulo 2

Fundamentos de Generación Text-to-Query y Consultas Híbridas

Este capítulo presenta los conceptos necesarios para comprender la propuesta del presente trabajo la cual será desarrollada en el Capítulo 4. La exposición avanza de manera progresiva, desde los fundamentos de los modelos de lenguaje hasta los mecanismos que permiten controlar y verificar las consultas generadas.

En primer lugar, se describen los LLMs, su forma de generar texto y los tipos de alucinaciones que pueden aparecer en este proceso (Sección 2.1). Luego, se revisan los modelos de datos relacional y de grafos de propiedad, así como su integración mediante el estándar SQL/PGQ (Sección 2.2) y se introduce el problema de *semantic parsing* y, dentro de este, la tarea de *text-to-query* (Sección 2.3).

Posteriormente, se explica el papel de una representación intermedia tipada como puente entre la interpretación realizada por el modelo y la generación final de una consulta ejecutable (Sección 2.4) así como se presentan las gramáticas formales y la decodificación restringida como mecanismos para guiar la salida del LLM hacia estructuras válidas (Sección 2.5). Finalmente, se revisan las formas de verificación estática y dinámica que permiten evaluar la corrección de la consulta generada (Sección 2.6).

El capítulo concluye con una síntesis (Sección line:10 **??**) que relaciona estos elementos y muestra cómo sirven de base para el análisis del estado del arte y para la propuesta de esta investigación.

2.1. Modelos de lenguaje y generación estructurada

Los LLMs, o modelos extensos de lenguaje, son modelos basados en arquitecturas *transformer* [Vaswani et al., 2017] y entrenados sobre grandes volúmenes de texto. En los últimos años, estos modelos se han convertido en una base importante para la investigación en tareas de *text-to-query*, esto debido a su capacidad para interpretar preguntas en lenguaje natural y generar respuestas en formatos estructurados.

Para el desarrollo de esta tesis, su relevancia se entiende a partir de tres aspectos principales: En primer lugar, los LLMs pueden adaptarse a esquemas que no fueron vistos durante el entrenamiento, especialmente mediante aprendizaje en contexto [Brown et al., 2020]. Esto permite aplicar el modelo a nuevos dominios sin requerir necesariamente un proceso completo de reentrenamiento. En segundo lugar, el uso de vocabularios basados en *subwords* les permite representar identificadores propios de un esquema, como nombres de tablas, columnas o relaciones, incluso cuando estos no aparecen literalmente en sus datos de entrenamiento. En tercer lugar, la disponibilidad de modelos preentrenados de propósito general permite distintas formas de adaptación, ya sea mediante *fine-tuning*, diseño del *prompt* o una combinación de ambos enfoques.

Sin embargo, esta capacidad de generación no resuelve por sí sola el problema de *text-to-query*, ya que los LLMs generan texto de forma autorregresiva, es decir, producen una secuencia de *tokens* prediciendo cada nuevo elemento a partir de los anteriores. Este proceso está guiado por patrones estadísticos aprendidos durante el entrenamiento, pero no incorpora de manera nativa las reglas gramaticales del lenguaje de consulta ni las restricciones del esquema sobre el cual se debe operar, dando como resultado que el modelo pueda producir consultas que parezcan correctas desde el punto de vista del lenguaje natural, pero que contienen errores estructurales, referencias inexistentes o interpretaciones equivocadas del esquema. Estos errores forman parte de los fenómenos que la literatura denomina *alucinaciones* [Ji et al., 2023]. Por ello, las siguientes subsecciones revisan tres aspectos necesarios para comprender este problema: la generación autorregresiva en modelos *transformer*, los principales regímenes de adaptación a la tarea y una tipología de alucinaciones específica para escenarios de *text-to-query* (§ line:20 **??**).

2.1.1. Arquitectura transformer y decodificación autorregresiva

La arquitectura *transformer* [Vaswani et al., 2017] permite procesar una secuencia de entrada $x = (x_1, \ldots, x_n)$ y representar cada *token* mediante vectores contextualizados. Para ello utiliza mecanismos de atención, los cuales permiten que el modelo relacione distintos elementos de la secuencia, incluso cuando se encuentran alejados entre sí. A diferencia de arquitecturas recurrentes, este procesamiento no depende de recorrer la secuencia paso a paso, sino de comparar los *tokens* entre sí mediante capas de atención.En tareas de *text-to-query*, los LLMs suelen emplearse como modelos generativos autorregresivos. Esto significa que generan la salida $y = (y_1, \ldots, y_m)$ un *token* a la vez, donde cada nuevo *token* depende de los generados previamente y de la entrada original. Formalmente, esta generación puede expresarse como:

$$P(y \mid x) \;=\; \prod_{t=1}^{m} P(y_t \mid y_{<t},\, x),$$

donde $P(y_t \mid y_{<t}, x)$ representa la probabilidad del siguiente *token* dado el prefijo ya generado $y_{<t}$ y la entrada x. En la práctica, esta probabilidad se obtiene mediante una capa *softmax* sobre el vocabulario del modelo. A partir de esa distribución, el siguiente *token* puede seleccionarse mediante estrategias como la maximización local o el muestreo.Esta forma de generación tiene dos consecuencias importantes para *text-to-query*. En primer lugar, el modelo decide cada paso en función de la probabilidad del siguiente *token*, pero no garantiza por sí mismo que la consulta completa sea válida. Por ejemplo, una elección

localmente probable puede introducir el nombre de una columna inexistente o una relación incorrecta entre tablas. En segundo lugar, el espacio de salida del modelo está definido por *tokens*, no por las reglas formales del lenguaje de consulta. Por ello, aunque el modelo pueda producir una secuencia que se parezca a una consulta válida, no existe una garantía nativa de que dicha secuencia respete la gramática del lenguaje ni las restricciones del esquema objetivo.

2.1.2. Prompting y aprendizaje en contexto

Sobre esta base de generación, la aplicación a tareas específicas admite dos regímenes complementarios. El *fine-tuning* supervisado modifica los parámetros del modelo mediante pares (pregunta, consulta) con el fin de sesgar $P(y \mid x)$ hacia el lenguaje objetivo. El *aprendizaje en contexto* [Brown et al., 2020], por su parte, mantiene los parámetros congelados y condiciona la generación en un *prompt* que incluye instrucciones y, opcionalmente, ejemplos de la tarea; en este régimen, la distribución efectiva de generación depende de la composición del *prompt*, no de una actualización del modelo.

Ambos regímenes comparten una limitación estructural: la salida es, en último término, el resultado de un muestreo sobre una distribución que desconoce las restricciones formales del lenguaje de consulta y del esquema. Por ello, el estudio de la fidelidad estructural de la salida requiere mecanismos externos al propio LLM.

2.2. Modelos de datos y lenguajes de consulta

Las consultas estudiadas en esta investigación se formulan sobre dos modelos de datos: el modelo relacional y el modelo de grafos de propiedad. Aunque ambos pueden representar información relacionada, lo hacen mediante estructuras distintas y con lenguajes de consulta diferentes. Por ello, esta sección presenta primero cada modelo por separado y luego explica su integración mediante el estándar SQL/PGQ.

2.2.1. Modelo relacional y álgebra relacional

El modelo relacional [Codd, 1970] representa los datos mediante *relaciones*: conjuntos de tuplas de aridad fija, donde cada posición se asocia a un atributo con dominio de valores. Formalmente, una relación R sobre el esquema $(A_1, \ldots, A_k)$ con dominios $(D_1, \ldots, D_k)$ se define como

$$R \subseteq D_1 \times \cdots \times D_k.$$

Un *esquema relacional* $\mathcal{S}_{\text{rel}}$ es un conjunto finito de tales relaciones, junto con restricciones de integridad: claves primarias, claves foráneas, unicidad y restricciones de dominio.

Sobre este modelo opera el álgebra relacional, cuyos operadores básicos $\{\sigma, \pi, \bowtie, \cup, -, \rho\}$, que corresponden respectivamente a selección, proyección, unión natural, unión, diferencia y renombramiento, son cerrados sobre el conjunto de relaciones y componen un

lenguaje declarativo con expresividad equivalente al cálculo relacional de primer orden con dominio acotado (*safe range calculus*). Esta equivalencia es metodológicamente relevante: propiedades como la existencia de una columna referenciada o la consistencia de una unión respecto a las claves foráneas admiten formalización como predicados decidibles sobre el álgebra, lo que habilita la verificación estática en tiempo lineal sobre el tamaño de la consulta.

2.2.2. SQL como lenguaje declarativo

SQL (*Structured Query Language*), estandarizado en la serie ISO/IEC 9075, es el lenguaje declarativo dominante para consultar bases de datos relacionales. Una consulta en forma `SELECT-FROM-WHERE-GROUP BY-HAVING-ORDER BY` se corresponde, con matices sintácticos, con una expresión del álgebra relacional extendida con agregaciones y ordenamiento. La distinción entre la consulta declarativa y el plan físico ejecutado por el motor es relevante para esta tesis: el objetivo de la generación es producir una expresión declarativa bien formada y grounded al esquema; la optimización y ejecución son responsabilidad del RDBMS.

2.2.3. Grafos de propiedad y pattern matching

Un *grafo de propiedad* [Angles et al., 2017] se define como una estructura

$$G \;=\; (V,\, E,\, \lambda,\, \mathrm{src},\, \mathrm{tgt},\, \mathrm{prop}),$$

donde V es un conjunto finito de vértices, E un conjunto finito de aristas, $\lambda : V \cup E \to 2^{\Sigma_L}$ asigna a cada elemento un conjunto de etiquetas de un alfabeto Σ_L, $\mathrm{src}, \mathrm{tgt} : E \to V$ determinan los extremos de cada arista dirigida, y $\mathrm{prop} : (V \cup E) \times K \rightharpoonup D$ asocia a cada elemento y cada clave $k \in K$ un valor parcial en un dominio D.

El lenguaje de consulta natural sobre esta estructura es el *pattern matching*: una consulta se expresa como un patrón P, esto es, un grafo con variables en lugar de identificadores, y el resultado es el conjunto de homomorfismos de P en G compatibles con las etiquetas y con las restricciones declaradas sobre las propiedades. Como ilustración mínima, sobre un grafo en el que los vértices llevan la etiqueta `Person` y las aristas la etiqueta `KNOWS`, el patrón

$$(p_1{:}\texttt{Person})\ \texttt{KNOWS}\ (p_2{:}\texttt{Person})\ \texttt{KNOWS}\ (p_3{:}\texttt{Person})$$

denota la consulta "personas conectadas por dos saltos de `KNOWS`"; cada homomorfismo de este patrón en G instancia las variables p_1, p_2, p_3 en vértices concretos y se proyecta como una tupla del resultado.

Una clase particularmente estudiada son las *regular path queries* (**RPQ!**s (**RPQ!**s)), que especifican caminos mediante expresiones regulares sobre el alfabeto Σ_L; generalizan las consultas de vecindad y admiten evaluación en tiempo polinomial mediante construcciones basadas en autómatas [Angles et al., 2017]. Esta formalización permite, a efectos

de verificación, formular reglas de tipado análogas a las del modelo relacional: toda referencia en una consulta sobre grafos debe corresponder a una etiqueta, un tipo o una propiedad declarada en la especificación de G.

2.2.4. El estándar SQL/PGQ como integración

El estándar ISO/IEC 9075-16:2023 [ISO/IEC JTC 1/SC 32, 2023] introduce SQL/PGQ como extensión de SQL para consultar grafos de propiedad definidos sobre tablas relacionales existentes. Su construcción fundamental es la expresión `GRAPH_TABLE(graph MATCH pattern COLUMNS (...))`, que produce una relación derivada a partir de un patrón de grafo y admite composición directa con las cláusulas relacionales clásicas. La integración es conceptualmente significativa por dos razones. Primera, unifica dos modelos de datos formalmente distintos bajo un único lenguaje, con lo cual el problema de la generación *text-to-query* debe abarcar, simultáneamente, el álgebra relacional y el *pattern matching* sobre grafos. Segunda, introduce una nueva categoría de consistencia estructural: la coherencia entre los componentes relacional y de grafo de una misma consulta, que no se reduce a la validez de cada componente por separado.

2.3. Semantic parsing y generación *text-to-query*

2.3.1. *Semantic parsing*: definición y marco general

El *semantic parsing* [Zelle and Mooney, 1996] es la tarea de transformar un enunciado en lenguaje natural u en una representación formal z que pueda ser ejecutada sobre un entorno $\mathcal{E}$. Este entorno puede ser, por ejemplo, una base de datos, un grafo de conocimiento o incluso un sistema físico. El objetivo es que la ejecución de dicha representación formal, denotada como $z_{\mathcal{E}}$, produzca la respuesta esperada para la pregunta original.

La representación formal z puede adoptar distintas formas, dependiendo del dominio y del tipo de sistema. Puede tratarse de expresiones en λ-cálculo, lenguajes específicos de dominio, consultas SQL, consultas sobre grafos como *Cypher*, entre otros. En términos generales, el problema requiere relacionar tres componentes: el enunciado en lenguaje natural, el entorno sobre el cual se ejecutará la consulta y el mecanismo que traduce el primero en una representación formal válida.

2.3.2. *Text-to-SQL* como caso particular y extensión a *text-to-query*

La tarea *text-to-SQL* es un caso particular de *semantic parsing*. En este caso, la representación formal pertenece al lenguaje SQL y el entorno de ejecución es una base de datos relacional con un esquema $\mathcal{S}_{\text{rel}}$. La entrada del sistema puede expresarse como el par $(u, \mathcal{S}_{\text{rel}})$, donde u es la pregunta en lenguaje natural y $\mathcal{S}_{\text{rel}}$ es el esquema de la base de datos. La salida esperada es una consulta q que, al ejecutarse sobre la base de datos, produzca la respuesta correcta.

La presente tesis aborda una extensión de este problema hacia *text-to-query* sobre SQL/PGQ. En este escenario, la consulta no se limita únicamente a operaciones relacionales, sino que también puede incluir patrones de grafo y combinaciones entre ambos modelos. Por ello, la entrada se define como un par $(u, \mathcal{S})$, donde $\mathcal{S}$ representa un esquema dual:

$$\mathcal{S} \;=\; (\mathcal{S}_{\text{rel}},\, \mathcal{S}_{\text{grafo}}).$$

Aquí, $\mathcal{S}_{\text{rel}}$ corresponde al componente relacional, mientras que $\mathcal{S}_{\text{grafo}}$ describe el componente de grafo de propiedad. La salida esperada es una consulta capaz de combinar operadores SQL tradicionales con patrones de grafo definidos bajo SQL/PGQ.

2.3.3. *Schema linking* y *grounding* al esquema

Una subtarea central en los sistemas *text-to-query* es el *schema linking*. Esta consiste en identificar qué elementos del esquema son mencionados, directa o indirectamente, en la pregunta del usuario. Dichos elementos pueden ser tablas, columnas, relaciones, etiquetas de vértices, tipos de aristas o propiedades del grafo.

Formalmente, esta correspondencia puede representarse mediante una función:

$$\ell : \text{menciones}(u) \to 2^{\mathcal{S}},$$

donde cada mención del enunciado u se asocia con un subconjunto de elementos del esquema $\mathcal{S}$ que podrían corresponder a dicha mención.

El *grounding* al esquema se refiere a la propiedad de que toda referencia utilizada en la consulta generada debe estar vinculada a un elemento existente y relevante del esquema. En otras palabras, la consulta no debería introducir tablas, columnas, etiquetas, aristas o propiedades que no estén definidas en $\mathcal{S}$ o que no hayan sido identificadas como pertinentes para la pregunta.

Cuando este proceso falla, aparecen alucinaciones referenciales, como se describió en la Sección line:107 **??**. Por esta razón, un sistema orientado a reducir este tipo de errores debe incorporar mecanismos explícitos de *grounding*. En la práctica, esto suele realizarse mediante una fase previa a la generación, en la que se seleccionan y organizan los elementos del esquema que serán considerados por el modelo.

2.4. Representaciones intermedias

2.4.1. Noción formal de representación intermedia

Una *representación intermedia* (RI) es un lenguaje formal $\mathcal{L}_{\mathrm{RI}}$ dotado de una gramática $\mathcal{G}_{\mathrm{RI}}$, un sistema de tipos $\mathcal{T}_{\mathrm{RI}}$ y una función de compilación

$$\kappa : \mathcal{L}_{\mathrm{RI}} \to \mathcal{L}_{\mathrm{objetivo}}$$

hacia el lenguaje de consulta efectivo, que en esta tesis es SQL/PGQ. Su rol en un pipeline de generación es mediar entre la salida del componente estadístico y la consulta ejecutable: el modelo produce, o es guiado a producir, instancias bien tipadas de $\mathcal{L}_{\mathrm{RI}}$; el compilador κ genera determinísticamente la consulta final.

2.4.2. Propiedades deseables

Una RI adecuada para *text-to-query* satisface, idealmente, las siguientes propiedades:

Abstracción sintáctica. $\mathcal{L}_{\mathrm{RI}}$ suprime decisiones sintácticas irrelevantes del lenguaje objetivo, tales como los alias, el orden de cláusulas o la elección entre `JOIN` y subconsulta. Dos consultas objetivo semánticamente equivalentes admiten, idealmente, una única representación en $\mathcal{L}_{\mathrm{RI}}$.

Tipado decidible. La corrección estructural de una instancia se decide mediante reglas locales, sin ejecutar la consulta. Cada nodo declara los tipos de sus operandos, y las reglas de tipado se verifican composicionalmente.

Compilación determinista. κ es una función total sobre instancias bien tipadas y produce la misma salida para instancias estructuralmente idénticas. Esto aísla la indeterminación del modelo en el espacio de $\mathcal{L}_{\mathrm{RI}}$.

Expresividad adecuada. $\mathcal{L}_{\mathrm{RI}}$ cubre las construcciones del lenguaje objetivo relevantes para la tarea. En SQL/PGQ, esto exige soporte para operadores relacionales, patrones de grafo y su composición híbrida.

2.4.3. Separación entre generación estadística y compilación determinista

La introducción de una RI permite separar, en el pipeline, la generación estadística, que produce hipótesis de interpretación, de la compilación determinista, que produce la consulta ejecutable. Esta separación es metodológicamente relevante: las alucinaciones del modelo quedan confinadas al espacio de $\mathcal{L}_{\mathrm{RI}}$ y se detectan mediante verificación de tipado antes de que alcancen el motor de ejecución. La propuesta del Capítulo 4 instancia esta idea con una RI específica para consultas híbridas SQL/PGQ.

2.5. Gramáticas formales y decodificación restringida

2.5.1. Gramáticas formales y jerarquía de Chomsky

Una gramática formal [Chomsky, 1956] se define como una tupla $\mathcal{G} = (N, \Sigma, P, S)$, donde N es un conjunto finito de símbolos no terminales, Σ un alfabeto de terminales, P un conjunto de producciones y $S \in N$ el símbolo inicial. Los lenguajes de consulta de interés para esta tesis pertenecen, en su núcleo sintáctico, a la clase libre de contexto: tanto SQL como SQL/PGQ admiten descripciones mediante gramáticas libres de contexto. La correspondencia entre un identificador y su declaración en el esquema, en cambio, no es decidible por la gramática y requiere un análisis adicional sobre el contexto de tipos. Esta separación entre validez gramatical y validez referencial motiva la combinación de decodificación restringida por gramática con verificación por tipos, descritas a continuación.

La teoría del parsing [Aho et al., 2006] distingue entre estrategias *top-down* (análisis descendente, típicamente LL) y *bottom-up* (análisis ascendente, típicamente LR). Para el propósito de esta tesis, la propiedad relevante no es el tipo específico de parser, sino la posibilidad de realizar *parsing incremental*: validar un prefijo parcial de la secuencia de salida contra la gramática mientras la generación está en curso.

2.5.2. Decodificación restringida como acoplamiento generación–parser

La decodificación restringida acopla el proceso autorregresivo de un LLM a un parser incremental de $\mathcal{G}$. En cada paso t, el parser mantiene el conjunto $C_t \subseteq \Sigma$ de *tokens* que extienden el prefijo $y_{<t}$ sin violar la gramática, y la distribución de salida del modelo se restringe a C_t:

$$\tilde{P}(y_t \mid y_{<t},\, x) \;\propto\; P(y_t \mid y_{<t},\, x) \cdot 1[\, y_t \in C_t \,].$$

Bajo esta restricción, toda secuencia generada es, por construcción, sintácticamente válida respecto a $\mathcal{G}$. La propiedad es estructural, no estadística: no depende de la calidad del modelo. Sin embargo, la decodificación restringida no garantiza *grounding*: una secuencia puede ser sintácticamente válida y, al mismo tiempo, referenciar identificadores ausentes del esquema, lo que motiva la introducción de mecanismos de verificación complementarios.

2.5.3. Sistemas de tipos y reglas locales de validación

Un sistema de tipos [Pierce, 2002] añade a una gramática un conjunto de reglas de inferencia que determinan cuándo una expresión está bien tipada. Las reglas se expresan como juicios de la forma $\Gamma \vdash e : \tau$, que se leen: "bajo el contexto Γ, la expresión e tiene tipo τ". Un ejemplo canónico es la regla de aplicación:

$$\frac{\Gamma \vdash f : \tau_1 \to \tau_2 \qquad \Gamma \vdash a : \tau_1}{\Gamma \vdash f(a) : \tau_2}.$$

Aplicado a una RI para *text-to-query*, el contexto Γ codifica el esquema dual $(\mathcal{S}_{\text{rel}}, \mathcal{S}_{\text{grafo}})$, y las reglas de tipado expresan propiedades como: toda referencia a una columna debe corresponder a una tabla del contexto; toda proyección de propiedad debe apuntar a un tipo de vértice o arista declarado; la unión de dos bloques relacionales exige esquemas compatibles. La decidibilidad local de las reglas las hace adecuadas para verificación estática en tiempo lineal sobre el tamaño de la RI.

2.5.4. Alcance y límites formales

Las técnicas de decodificación restringida y verificación por tipos presentan un alcance preciso y límites igualmente precisos. Garantizan la validez sintáctica y la consistencia referencial, pero no deciden, en general, la fidelidad semántica de la consulta respecto a la intención del usuario, ya que esta última no admite especificación formal previa. La estrategia global de una arquitectura robusta consiste, entonces, en combinar estas garantías formales con mecanismos de verificación dinámica, que se describen en la sección siguiente.

2.6. Verificación de consultas generadas

2.6.1. Verificación estática

La verificación estática designa el análisis de una consulta, o de su RI, sin ejecutarla, mediante la aplicación de reglas decidibles sobre su estructura y sobre el esquema. En el contexto de *text-to-query*, los chequeos estáticos relevantes incluyen: existencia de tablas, columnas, etiquetas y propiedades referenciadas; consistencia de tipos en operadores y condiciones; satisfacibilidad trivial de restricciones, como detección de condiciones contradictorias evidentes; y, en consultas híbridas, coherencia entre los componentes relacional y de grafo.

Formalmente, la verificación estática puede modelarse como un conjunto de predicados $\Phi = \{\varphi_1, \ldots, \varphi_k\}$ sobre consultas y esquemas. Una consulta q se considera *estáticamente válida* respecto a un esquema $\mathcal{S}$ si se cumple $\bigwedge_{i=1}^{k} \varphi_i(q, \mathcal{S})$. La decidibilidad local de cada φ_i hace de la verificación estática un mecanismo de costo constante o lineal respecto al tamaño de q, y por ello resulta aplicable en un bucle de refinamiento sin comprometer la eficiencia global.

2.6.2. Verificación dinámica y oráculos parciales

La verificación dinámica ejecuta la consulta en un entorno controlado y analiza la respuesta del motor. El motor actúa como *oráculo parcial*: distingue consultas sintáctica y referencialmente válidas de aquellas que disparan errores de ejecución, pero no decide, en general, la corrección semántica de una consulta ejecutable. Las señales útiles en este régimen incluyen los errores tipados del motor, la cardinalidad anómala del resultado,

entendida como un resultado vacío cuando se espera contenido o viceversa, la presencia inesperada de valores nulos y las desviaciones respecto a respuestas de referencia cuando estas existen.

2.6.3. Testing metamórfico

El *testing metamórfico* [Chen et al., 1998] extiende la verificación dinámica más allá de los oráculos explícitos. La idea central consiste en identificar relaciones entre entradas y salidas que deben preservarse bajo transformaciones conocidas, denominadas *relaciones metamórficas*. En *text-to-query*, relaciones como la invariancia del resultado bajo renombramiento de alias, la inclusión del resultado de una consulta con `LIMIT` en el de la misma consulta sin dicha cláusula, o la preservación de cardinalidades bajo reformulaciones semánticamente equivalentes, ofrecen un mecanismo para detectar inconsistencias sin conocer de antemano la respuesta correcta. La violación de una relación metamórfica evidencia un error; su satisfacción, sin embargo, no certifica corrección, sino que acota el espacio de hipótesis de error.

2.6.4. Bucles de refinamiento

Un bucle de refinamiento combina las verificaciones anteriores en un proceso iterativo: una consulta que falla una verificación produce una señal estructurada que se retroalimenta al generador para intentar producir una consulta corregida. El diseño del bucle está gobernado por dos decisiones. La primera es la política de retroalimentación, que determina qué información derivada del fallo se entrega al generador y en qué forma: mensaje crudo del motor, descriptor categórico o sugerencia estructurada. La segunda es el criterio de terminación, que limita el número de iteraciones y decide cuándo declarar la pregunta intrínsecamente ambigua o no resoluble bajo el contexto disponible.

Capítulo 3

Arquitecturas y Enfoques Actuales en Text-to-Query

El capítulo anterior estableció los fundamentos conceptuales que sostienen la generación *text-to-query*: modelos de lenguaje y sus alucinaciones, representaciones intermedias, decodificación restringida, mecanismos de verificación y el estándar híbrido SQL/PGQ. Sobre esa base, el presente capítulo analiza el estado del arte en sistemas *text-to-query* avanzados. La revisión no se organiza de forma cronológica ni como un inventario de trabajos, sino en torno a un marco analítico común que permite contrastar propuestas heterogéneas bajo preguntas estructurales uniformes.

La literatura reciente presenta una dispersión notable de criterios de clasificación: las *surveys* disponibles agrupan los trabajos, en unos casos, por la arquitectura del modelo de lenguaje subyacente, en otros por la fase del *pipeline* que optimizan, y en otros por el tipo de *prompting* que emplean [Quamar et al., 2022]. Estas clasificaciones, aunque útiles, tienden a mezclar niveles de abstracción heterogéneos y dificultan la identificación sistemática de vacíos de investigación. Por ello, este capítulo adopta como eje analítico una taxonomía multidimensional que distingue de forma explícita entre el paradigma de aprendizaje, la arquitectura de orquestación, la estrategia de representación, los mecanismos de enriquecimiento contextual, el paradigma de razonamiento y las propiedades evaluativas objetivo. Sobre este marco, cada sección analiza una dimensión de la literatura, cierra con observaciones críticas y articula, al final del capítulo, la brecha de investigación que motiva la propuesta de la tesis.

3.1. Marco taxonómico de análisis

Antes de revisar trabajos concretos es necesario definir el criterio bajo el cual se los compara. La taxonomía utilizada en este capítulo responde a seis preguntas estructurales, formuladas de manera que cada una aísle un aspecto conceptualmente independiente de los sistemas *text-to-query*. Estas preguntas se enuncian a continuación.

1. **Paradigma de aprendizaje.** ¿Cómo se ajusta, o no, el modelo base a la tarea? Las

opciones principales son el aprendizaje en contexto con parámetros congelados, el *fine-tuning* supervisado, el pre-entrenamiento continual, el aprendizaje por refuerzo con señal de ejecución y las combinaciones híbridas de los anteriores.

2. **Arquitectura del sistema.** ¿Cómo se orquestan los componentes que producen la consulta? Las alternativas van desde sistemas monolíticos *end-to-end* hasta *pipelines* modulares, marcos multi-agente, arquitecturas agénticas con búsqueda y esquemas de generación-reordenamiento.

3. **Estrategia de representación.** ¿Cómo se codifica la entrada, qué representación intermedia se utiliza y cómo se decodifica la salida? Esta dimensión se descompone en tres sub-ejes que se examinan por separado.

4. **Mecanismos de enriquecimiento contextual.** ¿Qué información adicional, más allá de la pregunta y el esquema, se moviliza? Aquí se agrupan el *schema linking*, la recuperación de contenido de la base de datos, la integración de conocimiento externo y los mecanismos de auto-refinamiento.

5. **Paradigma de razonamiento.** ¿Qué estructura cognitiva explícita guía la generación? Las opciones incluyen generación directa, cadena de pensamiento, descomposición, exploración multi-camino y bucles iterativos con retroalimentación.

6. **Propiedades evaluativas objetivo.** ¿Qué propiedades prioriza explícitamente cada propuesta? Entre ellas se encuentran la corrección, la eficiencia del modelo y de la consulta generada, la robustez, la adaptabilidad de dominio, la escalabilidad a esquemas de gran tamaño y la interpretabilidad.

Este marco permite examinar cada trabajo como un vector de decisiones a lo largo de las seis dimensiones, en lugar de como un punto singular sobre un eje cronológico. Las secciones siguientes no revisan los seis ejes de forma independiente, sino que los utilizan como lente transversal para analizar cinco grandes familias de enfoques: *pipelines* por etapas con representación intermedia (Sección 3.2), sistemas multi-agente y arquitecturas agénticas con búsqueda (Sección 3.3), decodificación restringida y verificación post-generación (Sección 3.4), robustez del esquema en entornos empresariales (Sección 3.5) y generación de consultas híbridas sobre el estándar SQL/PGQ (Sección 3.6). Cada sección se cierra con una observación crítica breve; tras revisar los criterios de evaluación empleados en la literatura (Sección 3.7), el capítulo se cierra con una síntesis de vacíos argumentada (Sección 3.8). Como referencia transversal, el Cuadro 3.1 resume el vector taxonómico de los trabajos centrales discutidos a continuación.

3.2. Pipelines por etapas y representaciones intermedias

Una de las líneas más consolidadas de la literatura parte de una observación estructural: la generación monolítica de SQL a partir de texto libre obliga al modelo de lenguaje a resolver simultáneamente problemas heterogéneos, tales como desambiguar la pregunta, identificar las tablas y columnas relevantes, producir la estructura sintáctica correcta y ajustar los valores literales al contenido real de la base de datos. Los trabajos agrupados

Tabla 3.1: Vector taxonómico de los trabajos centrales revisados, proyectado sobre los seis ejes definidos en la Sección 3.1. La última fila ubica la propuesta de la presente tesis. Las celdas con guion ("—") indican que la dimensión no es objetivo declarado del trabajo o no es aplicable.

Trabajo	**Aprendizaje**	**Arquitectura**	**Representación**	**Enriquecimiento**	**Razonamiento**	**Propiedad eval.**
RAT-SQL	FT supervisado	Monolítico	Atención relacional	*Schema linking*	Directo	Corrección
IRNet	FT supervisado	Pipeline	SemQL	*Schema linking*	Directo	Corrección
RESDSQL	FT supervisado	Pipeline 2 etapas	*Skeleton*	*Schema linking*	Directo	Corrección
DIN-SQL	ICL	Pipeline 4 etapas	Plantilla SQL	*Schema linking*	Descomp. + auto-corr.	Corrección
MAC-SQL	ICL	Multi-agente fijo	Plantilla SQL	*Schema linking* + ejec.	Descomposición	Corrección
CHASE-SQL	ICL + selector FT	Agéntica con búsqueda	Múltiples cadenas	Ejec. + selector	Multi-camino + *reranking*	Corrección
Alpha-SQL	ICL + MCTS	Agéntica con búsqueda	Estados parciales	Ejecución	Búsqueda explícita	Corrección
PICARD	FT + decod. restringida	Monolítico + parser	—	—	Directo restringido	Validez sintáctica
TA-SQL	ICL	Pipeline	*Alignment* interm.	*Schema linking*	Pre-alineamiento	Mitigación de alucin.
CRED-SQL	ICL	Pipeline	EDL ejecutable	Recuperación por *clusters*	Descomposición	Escalabilidad
STRuCT-LLM	RL (GRPO)	Monolítico	—	Ejecución (SQL+Cypher)	Directo	Razonamiento dual
SQLHD	ICL	Pipeline + verificación	—	—	Auto-refinamiento metamórfico	Detección de alucin.
BLENDSQL	ICL	Pipeline	SQL extendido	Recup. texto+tabla	Directo	Cobertura híbrida
SchemaGraphSQL	ICL + grafos	Pipeline	—	Grafo del esquema	Búsqueda de camino	Escalabilidad
Propuesta (esta tesis)	ICL + verificación	Pipeline tipado	RI tipada SQL/PGQ	*Schema linking* + *grounding*	Restringido + verif. iter.	Validez y *grounding* híbrido

en esta línea descomponen el problema en etapas, con frecuencia articuladas alrededor de una representación intermedia, y asignan cada etapa a un módulo especializado.

El trabajo precursor de esta tradición es IRNet [Guo et al., 2019], que introduce SemQL como representación intermedia que abstrae las particularidades sintácticas de cada dialecto SQL y separa la interpretación semántica del *binding* concreto al esquema. La generalización de esta idea en la era **PLM!** (**PLM!**) se refleja en RAT-SQL [B. Wang and et al., 2021], que incorpora atención consciente de relaciones para codificar conjuntamente la pregunta y el esquema, y en RESDSQL [H. Li and Chen, 2023], que desacopla el *schema linking* del *skeleton parsing*, codifica los elementos del esquema mediante un ranker y genera primero una plantilla estructural (*skeleton*) antes de producir la consulta final. Trabajos posteriores refuerzan esta línea desde ángulos complementarios: Graphix-T5 [Li and et al., 2022] integra capas conscientes del grafo del esquema en un modelo T5; BRIDGE [Lin and et al., 2020] introduce *anchor texts* que enlazan valores de la pregunta con celdas del esquema; y SmBoP [Rubin and Berant, 2021] propone un decodificador semi-autorregresivo ascendente que construye sub-árboles sintácticos en paralelo, reduciendo la complejidad de la decodificación a un régimen logarítmico en la profundidad del árbol.

En la transición hacia modelos de gran escala, la representación intermedia se reinterpreta como artefacto de *prompting*. DIN-SQL [Pourreza and Rafiei, 2023] reorganiza el proceso en cuatro etapas (clasificación, descomposición, generación y autocorrección) y se convierte en referencia obligada para cualquier propuesta modular posterior. MAC-SQL [B. Wang and et al., 2023] lleva esta descomposición al nivel arquitectónico al asignar cada responsabilidad a un agente distinto: un *Selector* filtra el esquema, un *Decomposer* fragmenta la pregunta en sub-consultas y un *Refiner* valida la ejecución. DEA-SQL [Xie and et al., 2024] formaliza esta idea como un paradigma de *workflow*, mientras que DTS-SQL [Pourreza and Rafiei, 2024] demuestra que la descomposición es especialmente útil cuando el modelo base es pequeño, ya que permite ajustar cada sub-modelo a una sub-tarea más acotada.

Una variante conceptualmente distinta consiste en introducir representaciones intermedias específicas para razonamiento semántico o estructural, en lugar de plantillas SQL. CHESS [S. Talaei and et al., 2024] estructura el proceso como una cadena de módulos con razonamiento contextual; TA-SQL [Qu et al., 2024] introduce una fase de *align-*

ment previa a la generación con el propósito explícito de mitigar alucinaciones; SteinerSQL2025 [Xutao Mao, 2025] propone una descomposición matemática basada en árboles de Steiner para guiar el razonamiento; CRED-SQL [Shaoming Duan, 2025] introduce un *Execution Description Language* como representación intermedia ejecutable, orientada específicamente a bases de datos de gran escala; y Gen-SQL [J. Shi and et al., 2025] propone un esquema pseudo-intermedio que media entre la pregunta y el esquema real. Merecen mención aparte PARSQL [Yaxun Dai, 2025], que combina *parsing* y razonamiento en modelos pequeños con reordenamiento *best-of-n*, y CodeS [H. Li and et al., 2024], que adopta una estrategia opuesta al introducir pre-entrenamiento continual sobre corpus específicos de SQL, trasladando la complejidad al modelo mismo y reduciéndola en el *pipeline* de inferencia.

Observación crítica. La descomposición en etapas ha producido mejoras sostenidas en los benchmarks cerrados como Spider y BIRD, pero al precio de una fragmentación metodológica considerable. Cada propuesta introduce su propia representación intermedia, con grados variables de formalización: desde gramáticas explícitas como SemQL y NatSQL, hasta plantillas semi-estructuradas y formatos ad hoc. Esta heterogeneidad dificulta comparaciones directas y, más importante, pocos trabajos reportan propiedades formales verificables sobre la representación intermedia que justifiquen su uso más allá del desempeño empírico. Existe, además, una tensión no resuelta entre la expresividad de la representación intermedia y su capacidad de compilarse de forma determinista a SQL válido respecto a un esquema concreto; esta tensión se agrava, como se argumentará más adelante, al introducir patrones de grafo.

3.3. Sistemas multi-agente y arquitecturas agénticas con búsqueda

Una evolución natural de los *pipelines* modulares consiste en sustituir el flujo secuencial predeterminado por una arquitectura en la que varios agentes especializados colaboran o compiten, bien mediante protocolos fijos de intercambio, bien mediante un controlador que decide dinámicamente el siguiente paso. La literatura reciente muestra una proliferación de este tipo de sistemas, con diferencias sustantivas que conviene explicitar para no colapsarlos bajo la misma etiqueta.

MAC-SQL [B. Wang and et al., 2023] es representativo de la variante con roles fijos: los tres agentes mencionados operan bajo un flujo predeterminado, y la colaboración consiste en el intercambio de representaciones parciales entre etapas. OpenSearch-SQL [X. Xie and Guo, 2025] extiende esta idea introduciendo un módulo de *Alignment* entre agentes y un mecanismo de *dynamic few-shot* que selecciona ejemplos de demostración según la pregunta. Por su parte, XiYan-SQL [Yingqi Gao, 2024] propone un *Multi-Generator Ensemble* en el que distintos generadores producen candidatos que un modelo de selección reordena, y combina aprendizaje en contexto con *fine-tuning* del componente selector. ROUTE [Y. Qin and et al., 2025] adopta una estrategia complementaria: en lugar de congelar los modelos y orquestar agentes sobre ellos, realiza *Multi-task Supervised Fine-Tuning* sobre cinco sub-tareas de *text-to-SQL* y después integra los modelos

ajustados bajo un esquema de *Multi-agent Collaborative Prompting*.

Un salto cualitativo aparece en las arquitecturas agénticas con búsqueda explícita, en las que el flujo de ejecución no está predeterminado sino que es controlado dinámicamente. CHASE-SQL [M. Pourreza and et al., 2025] ilustra este desplazamiento: genera candidatos de consulta mediante tres cadenas de razonamiento diferentes (una basada en divide y vencerás, otra en planes de ejecución y otra en ejemplos sintéticos ajustados a la instancia), aplica un *Query Fixer* guiado por ejecución y selecciona el candidato final mediante comparaciones pareadas con un selector ajustado. Más radical aún es Alpha-SQL [B. Li and Luo, 2025], que utiliza *Monte Carlo Tree Search* sobre estados parciales de razonamiento, con el modelo de lenguaje actuando como *Action Model* que propone expansiones del árbol.

En el espectro opuesto, trabajos como DAIL-SQL [D. Gao and et al., 2023] muestran que un sistema monolítico bien diseñado, con selección cuidadosa de demostraciones y *prompting* estructurado, puede alcanzar resultados competitivos sin necesidad de orquestación multi-agente. Otros trabajos exploran dimensiones específicas: PET-SQL [Li and et al., 2024] introduce un esquema de *cross-consistency* entre múltiples modelos de lenguaje; PURPLE [T. Ren and et al., 2024] recupera *skeletons* lógicos como ejemplos de *few-shot*; MetaSQL [Y. Fan and et al., 2024] formaliza el paradigma *generate-then-rank* con generación condicionada por metadatos; G^3R [Y. Xiang and et al., 2023] combina generación guiada por grafos con reordenamiento enriquecido con conocimiento de dominio; y N-best List Rerankers [L. Zeng and Hakkani-Tur, 2022] establece las bases del reordenamiento con T5 y decodificación restringida por PICARD. Más recientemente, STRuCT-LLM [Stoisser et al., 2025] propone unificar el razonamiento tabular y de grafos mediante aprendizaje por refuerzo con *Group-Relative Policy Optimization*, introduciendo una señal de recompensa basada en ejecución que se aplica tanto a SQL como a Cypher. Esto es especialmente relevante para la presente tesis, dado que anticipa la necesidad de métodos que operen sobre lenguajes de consulta heterogéneos de forma simultánea.

Observación crítica. La terminología asociada a estas arquitecturas es considerablemente imprecisa en la literatura. Etiquetas como "colaboración", "multi-agente" y "sistema agéntico" se aplican indistintamente a sistemas que difieren en aspectos esenciales, como el grado de dinamismo del flujo de control o el nivel de acoplamiento entre componentes. La distinción entre un *pipeline* modular con varias etapas y un sistema genuinamente multi-agente requiere, para no ser trivial, una operacionalización explícita; sin ella, la taxonomía se reduce a una etiqueta de márketing.

Un segundo problema transversal es la opacidad del balance costo-beneficio. CHASE-SQL, por ejemplo, moviliza simultáneamente tres modelos de frontera (GPT-4, Gemini y Claude) para generar candidatos, con lo cual resulta difícil atribuir la mejora observada al método propuesto frente al efecto de *ensembling* de modelos heterogéneos. En el mismo sentido, los mecanismos de búsqueda explícita como MCTS en Alpha-SQL presentan costos computacionales raramente reportados en detalle, lo cual limita su aplicabilidad en escenarios empresariales con restricciones de latencia.

3.4. Decodificación restringida y verificación post-generación

Una línea paralela a las anteriores aborda el problema desde la óptica del control formal de la salida. El supuesto subyacente es que las alucinaciones sintácticas pueden eliminarse de raíz si el decodificador consulta una gramática formal en cada paso, y que las alucinaciones semánticas pueden detectarse con mecanismos de verificación que operan sobre la consulta generada o sobre su ejecución.

El trabajo canónico en decodificación restringida es PICARD [Scholak and et al., 2022], que integra una gramática de SQL en el proceso autorregresivo para descartar continuaciones inválidas en tiempo real. La contribución es metodológicamente significativa porque desacopla la validez sintáctica de la confianza del modelo. Esta idea ha sido reutilizada en múltiples trabajos posteriores: RASAT [J. Qi and et al., 2022] combina PICARD con atención consciente de relaciones; TKK [C. Gao and et al., 2022] la utiliza dentro de un esquema de aprendizaje multi-tarea; y SHiP [L. Dou and et al., 2022] la adapta para el entrenamiento con datos sintéticos. Un refinamiento notable es SQLformer [A. Bazaga and et al., 2023], que propone generar directamente un grafo de consulta de forma auto-regresiva, unificando representación y decodificación en un solo proceso. En un registro más pragmático, CatSQL [H. Fu and et al., 2023] combina aprendizaje profundo con plantillas basadas en reglas para aplicaciones reales.

El complemento natural de la decodificación restringida es la verificación post-generación. Aquí la literatura se bifurca en dos aproximaciones que conviene distinguir con precisión. La primera es la verificación estática sobre una representación intermedia o sobre el SQL generado, que comprueba propiedades como la existencia de columnas referenciadas, la consistencia de claves foráneas en las uniones y la compatibilidad de tipos en las condiciones lógicas [B. Wang and et al., 2021]. La segunda es la verificación dinámica, que utiliza el RDBMS como oráculo parcial: se ejecuta la consulta en un entorno controlado y se analizan señales como errores de ejecución, valores nulos inesperados o cardinalidades anómalas. Trabajos como ErrorLLM [Hong et al., 2026] formalizan los tipos de error que pueden detectarse en este régimen, mientras que SQLHD [Yang et al., 2025] propone un esquema de detección de alucinaciones en dos etapas basado en relaciones metamórficas, en la línea conceptual de Chi [Chi, 2025].

Los mecanismos de auto-refinamiento constituyen una familia emparentada. DIN-SQL [Pourreza and Rafiei, 2023] incorpora un módulo de autocorrección *zero-shot*; TS-SQL [Wenbo Xu, 2025] propone un auto-refinamiento dirigido por casos de prueba generados; y SelECT-SQL [Z. Gu and et al., 2023] combina *ensembling* con autocorrección en una estrategia de *chain-of-thought*. En el ámbito del *prompting* especializado, ACT-SQL [Zhang and et al., 2023] genera cadenas de pensamiento automáticamente, ODIS [Chang and Fosler-Lussier, 2023] selecciona demostraciones en función del dominio, y C3-SQL [X. Dong and et al., 2023] formaliza un protocolo *zero-shot* con calibración y consistencia. En contextos multi-turno, CoE-SQL [H. Zhang and et al., 2024] introduce una cadena de ediciones que refina la consulta a lo largo del diálogo.

Observación crítica. La decodificación restringida garantiza la validez sintáctica pero resulta insuficiente, por construcción, frente a las alucinaciones semánticas: una consulta

puede ser sintácticamente impecable y, al mismo tiempo, referenciar una columna que no existe o plantear una unión inválida. La verificación dinámica cubre parcialmente este vacío, pero introduce sus propias limitaciones: un resultado vacío puede ser correcto o incorrecto, dependiendo de la pregunta, y la ejecución costosa no es viable en bucles de refinamiento agresivos sobre esquemas grandes. Además, los mecanismos de verificación actuales están diseñados para SQL puro; su extensión a consultas que combinan cláusulas relacionales y patrones de grafo aún no ha sido abordada de manera sistemática en la literatura revisada.

3.5. Robustez del esquema y entornos empresariales

Las tres líneas anteriores fueron evaluadas mayoritariamente sobre benchmarks cerrados como Spider [Yu et al., 2018] y BIRD [Li et al., 2023], los cuales, pese a su utilidad, representan escenarios idealizados respecto a los entornos empresariales reales. Una línea reciente de investigación pone el foco en la degradación del desempeño al desplazarse hacia esquemas masivos, con nomenclaturas complejas, datos ruidosos y flujos analíticos de múltiples pasos.

Spider 2.0 [Lei et al., 2025] es el benchmark más influyente en esta línea: documenta una caída drástica de los sistemas del estado del arte al enfrentarse a consultas empresariales reales, y formaliza métricas que capturan dimensiones ausentes en benchmarks previos. En el plano de los métodos, CRED-SQL [Shaoming Duan, 2025] aborda la escala mediante recuperación por *clusters* de esquema y una representación intermedia ejecutable; FinSQL [Zhang and et al., 2024] propone un marco específico para el dominio financiero; RSL-SQL [et al., 2024] introduce *schema linking* bidireccional con aumento contextual; CRUSH4SQL [et al., 2023] combina recuperación densa con alucinación controlada del esquema; y SchemaGraphSQL [Safdarian et al., 2025] aplica algoritmos de camino sobre el grafo del esquema para guiar el *schema linking*. Desde el lado de la robustez, el trabajo de Fürst et al. [Fürst et al., 2024] evalúa sistemáticamente la resiliencia frente a modelos de datos reales, y AmbiSQL [Ding et al., 2026] aborda la detección y resolución interactiva de ambigüedades en la pregunta del usuario.

La problemática de gobernanza y seguridad en entornos empresariales también ha recibido atención. SAFENLIDB [Liu et al., 2025] propone un marco de alineación para interfaces en lenguaje natural con preservación de privacidad, y LLM-FK [Tang et al., 2026] aborda la detección automática de claves foráneas en esquemas complejos como paso previo a una generación confiable. Desde la industria, el blog técnico de Wren AI [Chi, 2025] documenta prácticas de reducción de alucinaciones basadas en una capa semántica y en *blueprints* de consulta.

Observación crítica. Los trabajos revisados en esta sección muestran de forma consistente que el desempeño empírico sobre benchmarks cerrados sobreestima la capacidad real de los sistemas en entornos empresariales. Las propuestas concretas que abordan esta degradación (CRED-SQL, RSL-SQL, CRUSH4SQL, SchemaGraphSQL) intervienen sobre dimensiones parciales del problema —recuperación por *clusters*, enlazado bidireccional, alucinación controlada del esquema, búsqueda sobre el grafo del esquema—, pero

no se integran en un marco común que articule el *schema linking* con mecanismos de verificación posterior. Los componentes de gobernanza y seguridad, por su parte, son tratados como problemas adyacentes y rara vez se incorporan al *pipeline* de generación.

3.6. Generación de consultas híbridas sobre el estándar SQL/PGQ

La línea de investigación más cercana al objeto de esta tesis es la generación de consultas híbridas sobre esquemas que combinan el modelo relacional con patrones de grafo. El estándar ISO/IEC 9075-16:2023 formaliza la integración de SQL con PGQ [ISO/IEC JTC 1/SC 32, 2023 herramientas como DuckPGQ [Wolde et al., 2023] demuestran que los patrones de grafo pueden compilarse a SQL puro dentro de un motor relacional, con lo que la ejecución se vuelve tratable. La literatura específicamente orientada a la generación *text-to-query* sobre este estándar es, sin embargo, aún incipiente.

El trabajo más directamente alineado es STRuCT-LLM [Stoisser et al., 2025], que unifica razonamiento tabular y de grafos mediante aprendizaje por refuerzo con señal de ejecución, aplicada simultáneamente a SQL y Cypher. También son relevantes BLENDSQL [et al., 202 que extiende SQL para unificar la respuesta a preguntas híbridas sobre datos tabulares y texto no estructurado, y SGU-SQL [Qinggang Zhang, 2025], que introduce una codificación basada en grafos del esquema relacional como apoyo a la generación. Recientemente, Text2GQL-Bench [Lyu et al., 2026] se propone como primer benchmark sistemático de generación a lenguajes de consulta sobre grafos, aunque no incluye consultas que integren simultáneamente los paradigmas relacional y de grafo bajo un estándar común como SQL/PGQ.

Observación crítica. Los trabajos revisados comparten un patrón estructural: o bien tratan los grafos como un lenguaje separado (Cypher, GQL), o bien unifican paradigmas heterogéneos pero no específicamente el definido por el estándar SQL/PGQ. Ningún sistema de los identificados en esta revisión incorpora una representación intermedia tipada que cubra simultáneamente operadores relacionales y patrones de grafo, ni mecanismos de verificación que contemplen la coherencia cruzada entre ambos componentes. La intersección con la línea anterior —robustez del esquema bajo condiciones empresariales— queda, por consiguiente, sin abordar de forma sistemática.

3.7. Criterios de evaluación y señales empíricas

La comparación entre los trabajos revisados está mediada por los benchmarks y métricas empleados. Spider [Yu et al., 2018] y BIRD [Li et al., 2023] se han establecido como referencias de facto para evaluar corrección; CoSQL [Tao Yu, 2019a] y SParC [Tao Yu, 2019b] se utilizan para escenarios conversacionales; Spider 2.0 [Lei et al., 2025] introduce complejidad empresarial; y trabajos como SuperSQL [B. Li and et al., 2024] ofrecen análisis sistemáticos del estado del arte que son útiles para contextualizar las mejoras reportadas. Por su parte, benchmarks específicos como AmbiSQL [Ding et al., 2026] evalúan la capa-

cidad de detectar ambigüedad, mientras que Text2GQL-Bench [Lyu et al., 2026] abre la evaluación hacia lenguajes de consulta sobre grafos.

Las métricas dominantes (*Execution Accuracy*, *Exact Match* y, más recientemente, *Valid Efficiency Score*) capturan la corrección y la eficiencia ejecutiva, pero dejan parcialmente fuera propiedades como la verificabilidad estructural, la interpretabilidad del razonamiento y la trazabilidad de las decisiones intermedias. Estas propiedades son especialmente relevantes en escenarios empresariales, donde la auditoría de la consulta generada constituye un requisito operativo.

Desde el lado industrial, documentos como el de Adedeji y Mabere [Adedeji and Mabere, 2023], el estudio de Pérez et al. [Pérez et al., 2025] y la revisión de interfaces en lenguaje natural de Quamar et al. [Quamar et al., 2022] enmarcan estos requisitos y documentan la distancia entre el desempeño de investigación y las exigencias reales de producción. Esta brecha contextualiza la necesidad de propiedades evaluativas complementarias que la presente tesis busca abordar.

Observación crítica. El espacio evaluativo de la literatura revisada está fuertemente sesgado hacia métricas de corrección ejecutiva sobre benchmarks fijos. Propiedades estructurales como la verificabilidad de la consulta antes de su ejecución, la trazabilidad de las decisiones intermedias del modelo y la sensibilidad a perturbaciones controladas del esquema reciben un tratamiento marginal, pese a su relevancia para escenarios empresariales auditables. Esta carencia metodológica es coherente con la fragmentación observada en las secciones previas: sin un marco evaluativo común para propiedades estructurales, la comparación entre arquitecturas de mediación formal queda restringida al plano del desempeño agregado.

3.8. Síntesis, tensiones y vacíos de investigación

La revisión organizada en las secciones anteriores permite extraer un conjunto de observaciones estructurales que, consideradas en conjunto, delimitan el vacío de investigación que motiva la presente tesis.

Convergencia hacia la modularidad, con criterios heterogéneos. La gran mayoría de los trabajos revisados abandona la generación monolítica *end-to-end* en favor de alguna forma de modularidad, ya sea secuencial (RESDSQL, DIN-SQL), multi-agente (MAC-SQL, OpenSearch-SQL), o agéntica con búsqueda (Alpha-SQL, CHASE-SQL). Sin embargo, los criterios para descomponer el problema varían ampliamente, y no existe consenso sobre qué garantías estructurales debe proveer cada módulo. En particular, son pocos los trabajos que formulan propiedades verificables sobre la representación intermedia; con frecuencia, la descomposición se justifica por el desempeño empírico sin caracterizar sus invariantes.

Dualidad persistente entre validez sintáctica y corrección semántica. La decodificación restringida resuelve el problema de la validez sintáctica, pero no cierra el ciclo de la corrección semántica. Los mecanismos de verificación estática existentes operan típicamente sobre plantillas SQL genéricas y sobre propiedades básicas del esquema relacional. Los mecanismos dinámicos, por su parte, dependen del RDBMS como oráculo y son costosos a escala. Ninguno de los dos regímenes ha sido extendido de manera sistemática a un espacio de consultas donde coexistan estructuras relacionales y patrones de grafo.

Fragilidad ante el desplazamiento hacia esquemas empresariales. Los trabajos que evalúan explícitamente sobre Spider 2.0 y escenarios reales documentan caídas significativas de desempeño respecto a los benchmarks cerrados. Esto sugiere que las mejoras reportadas sobre Spider y BIRD no se transfieren de forma directa a entornos con esquemas masivos, nomenclaturas complejas y datos ruidosos. Las propuestas específicamente orientadas a esta problemática (CRED-SQL, RSL-SQL, CRUSH4SQL) abordan aspectos parciales (recuperación por *clusters*, enlazado bidireccional, alucinación controlada de esquema), pero no se integran en un marco común.

Tratamiento incipiente de consultas híbridas bajo un estándar común. La formalización del estándar SQL/PGQ [ISO/IEC JTC 1/SC 32, 2023] ofrece un sustrato único para consultas que combinan paradigmas relacional y de grafo dentro del mismo entorno. La literatura ha explorado separadamente la generación de SQL y la generación a lenguajes sobre grafos; propuestas como STRuCT-LLM [Stoisser et al., 2025] y BLENDSQL [et al., 2025] representan pasos hacia la unificación, pero no abordan específicamente el estándar híbrido ISO/IEC. La extensión de representaciones intermedias, gramáticas restringidas y mecanismos de verificación al dominio dual relacional/grafo permanece como un problema abierto.

Desatención a propiedades no medibles en benchmarks estándar. Las métricas dominantes reflejan principalmente la corrección ejecutiva. Propiedades como la trazabilidad del razonamiento, la verificabilidad estructural de la consulta generada, la sensibilidad a perturbaciones del esquema y la gobernanza de los datos reciben un tratamiento marginal en la literatura, a pesar de ser centrales en aplicaciones empresariales [Adedeji and Mabere, 2023, Quamar et al., 2022, Liu et al., 2025].

Ubicación de la presente tesis. Las observaciones anteriores convergen en una brecha articulada: *no existe, en la literatura revisada, una arquitectura que articule de forma sistemática una representación intermedia tipada, un mecanismo de generación restringido por gramática y un bucle de verificación estático y dinámico, diseñado específicamente para consultas híbridas sobre el estándar* SQL/PGQ, *con foco en la mitigación de alucinaciones estructurales y semánticas en entornos empresariales.* Las piezas individuales existen en la literatura; lo que falta es su articulación bajo un marco común que extienda las técnicas consolidadas del *text-to-SQL* al dominio dual del estándar híbrido. Cerrar esta brecha articulada constituye el propósito de la propuesta descrita en el Capítulo 4.

Capítulo 4

Propuesta: Arquitectura basada en Representación Intermedia para Generación Confiable de Consultas SQL/PGQ

Al concluir la revisión de la literatura actual, se evidencia una brecha de investigación claramente formulada: La literatura dispone de piezas maduras para abordar facetas individuales del problema *text-to-query*, entre ellas la descomposición del proceso de generación en etapas, la decodificación sujeta a restricciones, la verificación por ejecución y el enlazado robusto con el esquema.

No obstante, tales mecanismos han sido estudiados predominantemente de manera fragmentada, sin una articulación sistémica aplicada a consultas híbridas que combinen el paradigma relacional con patrones de grafo conforme al estándar SQL/PGQ [ISO/IEC JTC 1/SC 32, 202 A partir de esta constatación, el presente capítulo desarrolla la propuesta de investigación orientada a cerrar dicha brecha mediante una integración arquitectónica de estos aportes en el contexto de la generación de consultas empresariales sobre esquemas híbridos.

La propuesta se organiza en cinco bloques articulados entre sí. Primero, se establecen los requisitos y principios de diseño que debe satisfacer la arquitectura (Sección 4.1). Luego, se describe el flujo metodológico general del *pipeline*, así como la relación entre sus distintas etapas (Sección 4.2). A continuación, se detalla la arquitectura propuesta junto con sus componentes centrales, incluyendo la RI tipada, el anclaje al esquema, el compilador a SQL/PGQ y el bucle de verificación (Sección 4.3). Posteriormente, se presenta la estrategia de implementación sobre la base experimental seleccionada, distinguiendo los componentes reutilizados, los adaptados y los desarrollados específicamente para esta investigación (Sección 4.4). Por último, se fundamentan las decisiones de diseño adoptadas y se discuten las alternativas evaluadas (Sección 4.5).

4.1. Requisitos y principios de diseño

Los requisitos que se formulan en esta sección no constituyen una enumeración arbitraria, sino la traducción operacional de tres insumos previos del documento: (i) el planteamiento del problema desarrollado en el Capítulo 1, (ii) los fundamentos conceptuales revisados en el Capítulo 2, y (iii) las brechas identificadas en la síntesis del estado del arte. Cada requisito se identifica mediante un código (R_i) y se asocia con la dimensión del problema a la que responde.

R1. Anclaje estructural al esquema dual. La arquitectura debe producir consultas cuya estructura pueda verificarse contra un esquema dual compuesto por: (a) el esquema relacional, que incluye tablas, columnas, claves foráneas y tipos; y (b) el grafo de propiedad superpuesto, que comprende la definición de vértices, aristas, etiquetas y propiedades clave–valor conforme a la declaración `CREATE PROPERTY GRAPH` del estándar [ISO/IEC JTC 1/SC 32, 2023, Wolde et al., 2023]. Dicha verificabilidad debe ser posible antes de la ejecución de la consulta, y no solo como consecuencia de un error retornado por el motor.

R2. Representación intermedia tipada. La generación no debe producir SQL/PGQ de manera directa a partir del lenguaje natural. Debe mediar una RI explícita y tipada, cuya correctitud estructural sea decidible mediante reglas locales y cuya compilación a SQL/PGQ sea determinista. Este requisito traslada al contexto dual la lección central de IRNet [Guo et al., 2019] y RESDSQL [H. Li and Chen, 2023].

R3. Separación entre generación estadística y compilación determinista. La arquitectura debe preservar una separación estricta entre: (a) el componente estadístico encargado de producir hipótesis de interpretación a partir del lenguaje natural; y (b) el componente determinista responsable de compilar dichas hipótesis a SQL/PGQ. El propósito de esta separación es confinar las posibles alucinaciones del modelo al espacio de la RI, de modo que puedan detectarse y rechazarse antes de alcanzar el motor de ejecución.

R4. Verificación en dos regímenes. La propuesta debe incorporar un bucle de verificación que combine: (a) verificación estática sobre la RI y sobre la SQL/PGQ compilada, incluyendo referencias a elementos existentes, consistencia de tipos y coherencia entre los componentes relacionales y de grafo; y (b) verificación dinámica mediante ejecución controlada en un entorno de validación con DuckPGQ como motor de referencia [Wolde et al., 2023]. Este requisito retoma las propuestas de verificación revisadas en la Sección 3.4 y amplía su alcance al dominio híbrido.

R5. Modularidad y agnosticismo respecto del LLM. Los componentes de generación deben acoplarse al LLM mediante una interfaz abstracta que permita sustituir el modelo subyacente sin modificar la arquitectura general. Este requisito responde a la observación empírica de que los resultados reportados en el estado del arte dependen de manera crítica

del LLM utilizado, fenómeno documentado por SuperSQL [B. Li and et al., 2024] y por la proliferación de enfoques que combinan múltiples generadores [M. Pourreza and et al., 2025, Yingqi Gao, 2024].

R6. Trazabilidad de decisiones intermedias. Cada consulta generada debe poder rastrearse a lo largo de sus etapas intermedias: interpretación del lenguaje natural, construcción de la RI, compilación y verificación. La trazabilidad constituye un requisito previo para la interpretabilidad en entornos empresariales, una propiedad que el estado del arte actual aborda solo de manera marginal [Adedeji and Mabere, 2023, Quamar et al., 2022].

A partir de estos seis requisitos se derivan cinco *principios de diseño* que orientan las decisiones arquitectónicas concretas de la propuesta.

P1. Grounding antes que generación. Toda generación debe estar precedida por una fase explícita de anclaje al esquema dual. El modelo no genera SQL/PGQ sobre un esquema opaco, sino sobre una representación previamente proyectada y validada.

P2. Mediación obligatoria por representación intermedia. Ningún artefacto sintáctico de consulta alcanza el motor de ejecución sin pasar por la RI tipada. La RI constituye la frontera formal entre el componente estadístico y el componente determinista; la traducción desde la RI hacia SQL/PGQ es una función determinista, sin intervención adicional del modelo.

P3. Verificación en cascada. La validación de las consultas se organiza en niveles de costo creciente: la verificación estática precede a la dinámica, y solo las consultas que superan el nivel anterior son sometidas al siguiente. Esta progresión reduce el costo computacional y minimiza efectos colaterales sobre el RDBMS.

P4. Transparencia estructural. Cada componente debe exponer su entrada, su salida y su criterio de decisión en un formato auditable. De este modo, la opacidad propia del *prompting* monolítico se sustituye por una traza estructurada e inspeccionable.

P5. Desacoplamiento del modelo generador. El LLM se trata como una dependencia sustituible: los componentes de generación lo invocan a través de una interfaz abstracta, y ninguna decisión arquitectónica queda atada a un modelo específico.

Estos principios se concretan en el *pipeline* descrito en la siguiente sección.

4.2. Flujo metodológico general

El *pipeline* propuesto se articula en cinco etapas secuenciales, junto con un bucle de refinamiento asimétrico entre la construcción de la RI y la fase de verificación. La Figura 4.1 resume el flujo metodológico general de la propuesta.

1. **Pre-procesamiento y anclaje al esquema dual.** Esta etapa recibe como entrada la pregunta en lenguaje natural, el esquema relacional y la definición del grafo de

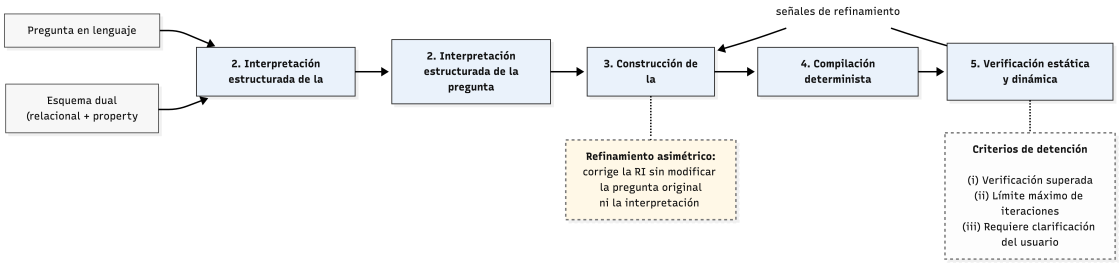

Figura 4.1: Flujo metodológico general del *pipeline* propuesto. Las cinco etapas secuenciales procesan la pregunta en lenguaje natural y el esquema dual (relacional y grafo de propiedad) hasta producir una consulta SQL/PGQ verificada.

propiedad. Su salida es un esquema proyectado, es decir, un subconjunto del esquema dual enriquecido con señales de relevancia, tales como tablas y columnas candidatas, vértices y aristas potencialmente implicados, así como *cell-values* relevantes. Esta fase materializa el principio P1 y se apoya en ideas de anclaje al esquema y reducción del espacio de búsqueda reportadas en [et al., 2024, S. Talaei and et al., 2024].

2. **Interpretación estructurada de la pregunta.** A partir de la pregunta y del esquema proyectado, esta etapa produce una descomposición explícita de la intención del usuario en componentes interpretables: entidades mencionadas, restricciones, proyecciones esperadas y tipo de razonamiento requerido. En particular, identifica si la consulta demanda razonamiento relacional, razonamiento sobre grafos o una combinación de ambos. Esta etapa se inspira en el paradigma de descomposición presentado en [Pourreza and Rafiei, 2023, B. Wang and et al., 2023], extendiéndolo al contexto híbrido SQL/PGQ.

3. **Construcción de la representación intermedia tipada.** Con base en la interpretación estructurada, se construye una instancia de la RI propuesta. Esta etapa constituye el núcleo conceptual de la arquitectura, pues traduce la intención del usuario a una forma estructurada, tipada y validable antes de cualquier compilación al lenguaje de consulta final.

4. **Compilación determinista a SQL/PGQ.** Una vez que la RI satisface sus restricciones estructurales, se compila a una consulta concreta mediante reglas deterministas, sin intervención adicional del LLM. De este modo, la generación estadística queda separada de la síntesis sintáctica final, y la consulta producida se ajusta por construcción a la gramática del estándar objetivo.

5. **Verificación estática y dinámica.** La consulta compilada se somete, en primer lugar, a una verificación estática orientada a comprobar su consistencia estructural respecto del esquema dual. Si supera esta fase, se procede a una verificación dinámica mediante ejecución controlada sobre DuckPGQ [Wolde et al., 2023]. Los errores detectados en cualquiera de estos dos niveles se transforman en señales de refinamiento que retroalimentan la etapa de construcción de la RI.

El bucle de refinamiento es asimétrico en el sentido que la retroalimentación no

modifica la pregunta original ni reabre la interpretación de alto nivel de la etapa dos, sino que actúa exclusivamente sobre la construcción de la RI. Esta decisión responde al principio P2, ya que concentra la corrección en el espacio donde la estructura puede manipularse de forma explícita y verificable mediante reglas. El refinamiento concluye cuando se satisface alguno de los siguientes criterios: (i) la verificación es superada, (ii) se alcanza un número máximo de iteraciones, o (iii) el sistema determina que la pregunta presenta una ambigüedad intrínseca que requiere clarificación del usuario, en consonancia con lo planteado por [Ding et al., 2026].

4.3. Arquitectura y componentes

4.4. Implementación sobre la base experimental

4.5. Justificación de decisiones de diseño

4.6. Síntesis y transición

Capítulo 5

Experimentos

Capítulo 6

Resultados

Capítulo 7

Conclusiones y Trabajos Futuros

Las conclusiones de la tesis son una parte muy importante y tiene las siguientes partes.

Escribir las conclusiones de su trabajo por cada uno de los objetivos específicos definidos en el Capítulo 1 y luego escribir la conclusión general de tu trabajo.

7.1. Problemas encontrados

La segunda parte de este capítulo corresponde a los problemas encontrados, esta sección es muy importante para que los siguientes estudiantes o investigadores que hagan algo en esta línea no cometan los mismos errores y tu tesis sea un buen peldaño para avanzar más rápido.

7.2. Recomendaciones

En esta sección el tesista debe reflejar que la tesis ha permitido adquirir nuevos conocimientos que podrían servir para guiar otros trabajos en el futuro.

7.3. Trabajos futuros

En base a los puntos anteriores es recomendable que tu tesis también sugiera trabajos futuros. Esta sección es esecialmente útil para otras ideas de tesis.

Todo este capítulo no debe ser más de unas 4 páginas.

Bibliografía

[A. Bazaga and et al., 2023] A. Bazaga, P. L. and et al. (2023). Sqlformer: Deep autoregressive query graph generation for text-to-sql translation.

[Adedeji and Mabere, 2023] Adedeji and Mabere, M. (2023). From data to decisions: Llms redefining business intelligence author. *Journal of Artificial Intelligence*, 6:14.

[Aho et al., 2006] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2 edition.

[Angles et al., 2017] Angles, R., Arenas, M., Barceló, P., Hogan, A., Reutter, J., and Vrgoč, D. (2017). Foundations of modern query languages for graph databases. *ACM Computing Surveys*, 50(5).

[B. Li and et al., 2024] B. Li, Y. Luo, C. C. and et al. (2024). The dawn of natural language to sql: Are we fully ready?

[B. Li and Luo, 2025] B. Li, J. Zhang, J. F. Y. X. C. C. N. T. and Luo, Y. (2025). Alpha-sql: Zero-shot text-to-sql using monte carlo tree search. *ICML 2025*.

[B. Wang and et al., 2023] B. Wang, C. Ren, J. Y. and et al. (2023). Mac-sql: A multi-agent collaborative framework for text-to-sql. *COLING 2025*.

[B. Wang and et al., 2021] B. Wang, R. Shin, X. L. and et al. (2021). Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers. *ACL 2020*.

[Brown et al., 2020] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[C. Gao and et al., 2022] C. Gao, B. Li, W. Z. and et al. (2022). Towards generalizable and robust text-to-sql parsing. *EMNLP 2022 Findings*.

[Chang and Fosler-Lussier, 2023] Chang, S. and Fosler-Lussier, E. (2023). Selective demonstrations for cross-domain text-to-sql. *Findings of EMNLP 2023*.

[Chen et al., 1998] Chen, T. Y., Cheung, S. C., and Yiu, S. M. (1998). Metamorphic testing: A new approach for generating next test cases. Technical Report HKUST-CS98-01, Department of Computer Science, Hong Kong University of Science and Technology.

[Chi, 2025] Chi, H. (2025). Reducing hallucinations in text-to-sql building trust and accuracy in data access.

[Chomsky, 1956] Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory*, 2(3):113–124.

[Codd, 1970] Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387.

[D. Gao and et al., 2023] D. Gao, H. Wang, Y. L. and et al. (2023). Text-to-sql empowered by large language models: A benchmark evaluation. *PVLDB 2024*.

[Ding et al., 2026] Ding, Z., Lin, Y., Zeng, T., Zhu, R., Ding, B., and Zhou, J. (2026). Ambisql: Interactive ambiguity detection and resolution for text-to-sql.

[et al., 2023] et al., M. K. (2023). Crush4sql: Collective retrieval using schema hallucination for text2sql. *EMNLP 2023*.

[et al., 2025] et al., S. G. (2025). Blendsql: A scalable dialect for unifying hybrid question answering in relational algebra. *ACL 2024 Findings*.

[et al., 2024] et al., Z. C. (2024). Rsl-sql: Robust schema linking in text-to-sql generation.

[Fürst et al., 2024] Fürst, J., Kosten, C., Nooralahzadeh, F., Zhang, Y., and Stockinger, K. (2024). Evaluating the data model robustness of text-to-sql systems based on real user queries.

[Guo et al., 2019] Guo, J., Zhan, Z., Gao, Y., Xiao, Y., Lou, J.-G., Liu, T., and Zhang, D. (2019). Towards complex text-to-sql in cross-domain database with intermediate representation.

[H. Fu and et al., 2023] H. Fu, C. Liu, B. W. and et al. (2023). Catsql : Towards real world natural language to sql applications. *PVLDB 2023*.

[H. Li and Chen, 2023] H. Li, J. Zhang, C. L. and Chen, H. (2023). Resdsql: Decoupling schema linking and skeleton parsing for text-to-sql. *AAAI 2023*.

[H. Li and et al., 2024] H. Li, J. Zhang, H. L. and et al. (2024). Codes: Towards building open-source language models for text-to-sql. *SIGMOD 2024*.

[H. Zhang and et al., 2024] H. Zhang, R. Cao, H. X. and et al. (2024). Coe-sql: In-context learning for multi-turn text-to-sql with chain-of-editions. *NAACL 2024*.

[Hong et al., 2026] Hong, Z., Chen, H., Yuan, Z., Zhang, Q., Zhuang, L., Liao, Q., Huang, F., Song, Y., and Huang, X. (2026). Errorllm: Modeling sql errors for text-to-sql refinement.

[ISO/IEC JTC 1/SC 32, 2023] ISO/IEC JTC 1/SC 32 (2023). Information technology – database languages – sql – part 16: Property graph queries (sql/pgq). Standard, International Organization for Standardization, Geneva, CH.

[J. Qi and et al., 2022] J. Qi, J. Tang, Z. H. and et al. (2022). Rasat: Integrating relational structures into pretrained seq2seq model for text-to-sql. *EMNLP 2022*.

[J. Shi and et al., 2025] J. Shi, Bo Xu, J. L. and et al. (2025). Gen-sql: Efficient text-to-sql by bridging natural language question and database schema with pseudo-schema. *COLING 2025*.

[Ji et al., 2023] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Madotto, A., and Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12).

[L. Dou and et al., 2022] L. Dou, Y. Gao, X. L. and et al. (2022). Importance of synthesizing high- quality data for text-to-sql parsing.

[L. Zeng and Hakkani-Tur, 2022] L. Zeng, S. H. K. P. and Hakkani-Tur, D. (2022). N-best hypothe- ses reranking for text-to-sql systems.

[Lei et al., 2025] Lei, F., Su, J., Chen, Y., Li, Q., Zhang, Z.-X., et al. (2025). Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. In *Proceedings of the 13th International Conference on Learning Representations (ICLR).* Oral Presentation.

[Li and et al., 2022] Li, J. and et al. (2022). Graphix-t5: Mixing pre-trained transformers with graph-aware layers for text-to-sql parsing.

[Li et al., 2023] Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., Zhou, X., Ma, C., Li, G., Chang, K. C.-C., Huang, F., Cheng, R., and Li, Y. (2023). Can LLM already serve as a database interface? a BIg bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS).*

[Li and et al., 2024] Li, Z. and et al. (2024). Pet-sql: A prompt-enhanced two-round refinement of text-to-sql with cross-consistency.

[Lin and et al., 2020] Lin, X. V. and et al. (2020). Bridging textual and tabular data for cross-domain text-to-sql semantic parsing.

[Liu et al., 2025] Liu, R., Chen, X., Zhang, J., Zhang, Q., Zhang, Y., and Yang, B. (2025). Safenlidb: A privacy-preserving safety alignment framework for llm-based natural language database interfaces.

[Lyu et al., 2026] Lyu, S., Ban, L., Wu, Z., Luo, T., Liu, J., Ma, C., Luo, Y., Tang, N., Qi, S., Lin, H., Liu, Y., and Hong, C. (2026). Text2gql-bench: A text to graph query language benchmark [experiment, analysis & benchmark].

[M. Pourreza and et al., 2025] M. Pourreza, H. Li, R. S. and et al. (2025). Chase-sql: Multi-path reasoning and preference optimized candidate selection in text-to-sql. *ICLR 2025.*

[Pérez et al., 2025] Pérez, A. S., Boukhary, A., Papotti, P., Lozano, L. C., and Elwood, A. (2025). An LLM-based approach for insight generation in data analysis. In Chiruzzo, L., Ritter, A., and Wang, L., editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 562–582, Albuquerque, New Mexico. Association for Computational Linguistics.

[Pierce, 2002] Pierce, B. C. (2002). *Types and Programming Languages.* MIT Press.

[Pourreza and Rafiei, 2023] Pourreza, M. and Rafiei, D. (2023). Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *NeurIPS 2023.*

[Pourreza and Rafiei, 2024] Pourreza, M. and Rafiei, D. (2024). Dts-sql: Decomposed text-to-sql with small large language models. *EMNLP 2024 Findings*.

[Qinggang Zhang, 2025] Qinggang Zhang, Hao Chen, J. D. S. C. F. H. X. H. (2025). Structure guided large language model for sql generation. *ICML 2024*.

[Qu et al., 2024] Qu, G., Li, J., Li, B., et al. (2024). Before generation, align it! a novel and effective strategy for mitigating hallucinations in text-to-sql generation. *ACL 2024 Findings*.

[Quamar et al., 2022] Quamar, A., Efthymiou, V., Lei, C., and Lei, C. (2022). Natural language interfaces to data. *Foundations and Trends in Databases*, 11(4):319–414.

[Rubin and Berant, 2021] Rubin, O. and Berant, J. (2021). Smbop: Semi-autoregressive bottom-up semantic parsing. *NAACL 2021*.

[S. Talaei and et al., 2024] S. Talaei, M. Pourreza, Y.-C. C. and et al. (2024). Chess: Contextual harnessing for efficient sql synthesis. *ICML*.

[Safdarian et al., 2025] Safdarian, A., Mohammadi, M., Jahanbakhsh, E., Naderi, M. S., and Faili, H. (2025). Schemagraphsql: Efficient schema linking with pathfinding graph algorithms for text-to-sql on large-scale databases.

[Scholak and et al., 2022] Scholak, T. and et al. (2022). Picard: Parsing incrementally for constrained auto-regressive decoding from language models. *EMNLP 2021*.

[Shaoming Duan, 2025] Shaoming Duan, Zirui Wang, C. L. Z. Z. Y. Z. P. H. L. Y. Z. P. (2025). Cred-sql: Enhancing large scale database text-to-sql.

[Stoisser et al., 2025] Stoisser, J. L., Martell, M. B., Phillips, L., Hansen, C., and Fauqueur, J. (2025). Struct-llm: Unifying tabular and graph reasoning with reinforcement learning for semantic parsing. *ICLR 2025*.

[T. Ren and et al., 2024] T. Ren, Y. Fan, Z. H. and et al. (2024). Purple: Making a large language model a better sql writer. *ICDE 2024*.

[Tang et al., 2026] Tang, Z., Zhang, Y., Cai, S., and Wang, R. (2026). Llm-fk: Multi-agent llm reasoning for foreign key detection in large-scale complex databases.

[Tao Yu, 2019a] Tao Yu, Rui Zhang, H. Y. E. e. a. (2019a). Cosql: A conversational text-to-sql challenge towards cross-domain natural language interfaces to databases. *EMNLP 2019*.

[Tao Yu, 2019b] Tao Yu, Rui Zhang, M. Y. e. a. (2019b). Sparc: Cross-domain semantic parsing in context. *ACL 2019*.

[Vaswani et al., 2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[Wenbo Xu, 2025] Wenbo Xu, Haifeng Zhu, L. Y. C. L. P. H. S. D. J. Z. P. (2025). Ts-sql: Test-driven self-refinement for text-to-sql. *EMNLP 2025 Findings*.

[Wolde et al., 2023] Wolde, D. t., Szárnyas, G., and Boncz, P. (2023). Duckpgq: Bringing sql/pgq to duckdb. *Proc. VLDB Endow.*, 16(12):4034–4037.

[X. Dong and et al., 2023] X. Dong, C. Zhang, Y. G. and et al. (2023). C3: Zero-shot text-to-sql with chatgpt.

[X. Xie and Guo, 2025] X. Xie, G. Xu, L. Z. and Guo, R. (2025). Opensearch-sql: Enhancing text-to-sql with dynamic few-shot and consistency alignment. *ACM on Management of Data*.

[Xie and et al., 2024] Xie, Y. and et al. (2024). Decomposition for enhancing attention: Improving llm-based text-to-sql through workflow paradigm. *ACL 2024 Findings*.

[Xutao Mao, 2025] Xutao Mao, Tao Liu, H. Z. (2025). Steinersql: Graph-guided mathematical reasoning for text-to-sql generation.

[Y. Fan and et al., 2024] Y. Fan, Z. He, T. R. and et al. (2024). Metasql: A generate-then-rank framework for natural language to sql translation.

[Y. Qin and et al., 2025] Y. Qin, C. Chen, Z. F. and et al. (2025). Route: Robust multi-task tuning and collaboration for text-to-sql. *ICLR 2025*.

[Y. Xiang and et al., 2023] Y. Xiang, Q.-W. Zhang, X. Z. and et al. (2023). G3r: A graph-guided generate-and-rerank framework for complex and cross-domain text-to-sql generation. *ACL 2023 Findings*.

[Yang et al., 2025] Yang, B., Xia, Y., Sun, W., and Liu, Y. (2025). Hallucination detection for llm-based text-to-sql generation via two-stage metamorphic testing.

[Yaxun Dai, 2025] Yaxun Dai, Haiqin Yang, M. H. P. C. (2025). Parsql: Enhancing text-to-sql through parsing and reasoning. *ACL 2025 Findings*.

[Yingqi Gao, 2024] Yingqi Gao, Yifu Liu, X. L. X. S. Y. Z. Y. W. S. L. W. L. Y. H. Z. L. J. G. L. M. Y. L. (2024). Xiyan-sql: A multi-generator ensemble framework for text-to-sql.

[Yu et al., 2018] Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., and Radev, D. (2018). Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

[Z. Gu and et al., 2023] Z. Gu, J. Fan, N. T. and et al. (2023). Select-sql: Self-correcting ensemble chain-of-thought for text-to-sql.

[Zelle and Mooney, 1996] Zelle, J. M. and Mooney, R. J. (1996). Learning to parse database queries using inductive logic programming. In *Proceedings of the Thirteenth National Conference on Artificial Intelligence (AAAI)*, pages 1050–1055.

[Zhang and et al., 2024] Zhang, C. and et al. (2024). Finsql: Model-agnostic llms-based text-to-sql framework for financial analysis.

[Zhang and et al., 2023] Zhang, H. and et al. (2023). Act-sql: In-context learning for text-to-sql with automatically-generated chain-of-thought. *Findings of EMNLP 2023*.